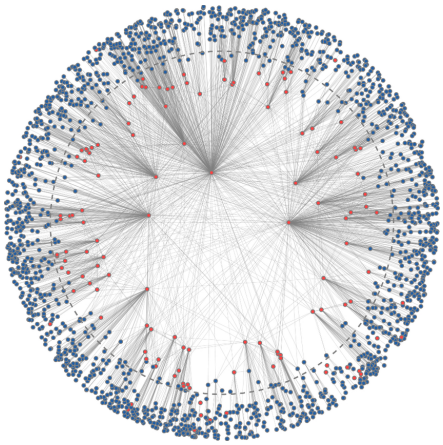
National Technical University of Athens
School of Electrical and Computer Engineering
IPP Data Science and Machine Learning

Combining Geometry and Learning: Hyperbolic Graph Embeddings with GNNs

Diploma Thesis

of

CHONTZAKIS P. DIONYSIOS



Supervisor: Symeon Papavassiliou
Professor, N.T.U.A.

Athens, July 2025

National Technical University of Athens
School of Electrical and Computer Engineering
IPP Data Science and Machine Learning

Combining Geometry and Learning: Hyperbolic Graph Embeddings with GNNs

Diploma Thesis

of

CHONTZAKIS P. DIONYSIOS

Supervisor: Symeon Papavassiliou
Professor, N.T.U.A.

Approved by the examination committee on 1st July 2025.

(Signature)	*(Signature)*	*(Signature)*
.....	.....	.....
Symeon Papavassiliou	George Matsopoulos	Eleni Stai
Professor, N.T.U.A.	Professor, N.T.U.A.	Assistant Professor, N.T.U.A.

Athens, July 2025

National Technical University of Athens
School of Electrical and Computer Engineering
IPP Data Science and Machine Learning

DISCLAIMER ON ACADEMIC ETHICS AND INTELLECTUAL PROPERTY RIGHTS

Being fully aware of the implications of copyright laws, I expressly state that this diploma thesis, as well as the electronic files and source codes developed or modified in the course of this thesis, are solely the product of my personal work and do not infringe any rights of intellectual property, personality and personal data of third parties, do not contain work / contributions of third parties for which the permission of the authors / beneficiaries is required and are not a product of partial or complete plagiarism, while the sources used are limited to the bibliographic references only and meet the rules of scientific citing. The points where I have used ideas, text, files and / or sources of other authors are clearly mentioned in the text with the appropriate citation and the relevant complete reference is included in the bibliographic references section. I fully, individually and personally undertake all legal and administrative consequences that may arise in the event that it is proven, in the course of time, that this thesis or part of it does not belong to me because it is a product of plagiarism.

(Signature)

.....
Dionysios Chontzakis

1st July 2025

Abstract

Graphs are Mathematical models used to represent conceptual connections between entities. In these models, entities are represented by nodes and the connections between them by edges. In the domain of Social Network Analysis, the analysis of these models primarily aims to highlight similar nodes and secondly to create communities of similar nodes that communicate, a process known as Community Detection. Representing graphs in Hyperbolic Spaces enriches the information they carry. This is because Hyperbolic Geometry better characterizes the Geometry of graphs with a hierarchical structure. In such Spaces, the similarity between nodes is translated in hyperbolic distance, which adds a spatial dimension in the community organization.

The problem at hand is how a network can be embedded into Hyperbolic Space. This can be achieved through various methods, including the well-known HyperMap approach. The aim of the present work was to develop a new graph embedding method using Machine Learning that approximates the embedding produced by HyperMap. Within this context, Graph Neural Network models were employed on synthetic graph data. Graphs with 100 or more nodes were embedded, and the embedding quality was compared with that of HyperMap. Additionally, the performance of the model was investigated as the size of the graphs increased.

Keywords

Graphs, Social Network Analysis, Graph Embedding, Machine Learning, HyperMap, Hyperbolic Spaces, Graph Neural Networks

to my parents

Acknowledgements

First of all, I would like to thank Professor Vasileios Karyotis for supervising this diploma thesis and for giving me the opportunity to carry it out in the Network Management and Optimal Design Laboratory. I would also like to especially thank Symeon Papavassiliou for his guidance and for the excellent collaboration we had. Finally, I would like to thank my parents for the guidance and moral support they have offered me throughout all these years.

Athens, July 2025

Dionysios Chontzakis

Table of Contents

List of Figures

List of Images

List of Tables

Preface

This diploma thesis was prepared within the framework of the Interdepartmental Postgraduate Program "Data Science and Machine Learning" of the School of Electrical and Computer Engineering at the National Technical University of Athens. The thesis was carried out at the Network Management & Optimal Design Laboratory of NTUA.

Cover image source: [10]

Chapter 1

Introduction

We live in an era in which every person's digital footprint is greater than at any other point in time. From the establishment of Social Media in people's lives, to the revolution of Artificial Intelligence and these models' need for data, digital information and its processing have become some of the most important research issues for today's researchers.

The development of graphs as Mathematical models, gives us the ability to capture the meaning of connection between data. For example, users of Social Media are entities for which other methods of data management and storage do not allow us to express the notion of the relationships between them. Therefore, for every system in which there is a need to define the concept of connection between entities, the use of graphs is the most appropriate method of analysis. Some relevant examples are the modeling of Social Media users (such as Facebook), the structure of the Internet and, specifically, the navigation among websites, as well as the structure of molecules in Biology (bonds between atoms).

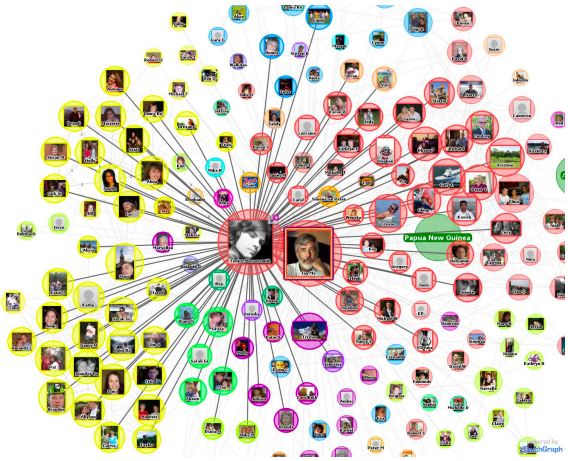


Illustration 1.1: *Example of modeling a real network (Facebook) as a graph. Source: [1]*

Through the analysis of these complex models, five dominant categories have emerged. These are defined as Regular, Random, Random Geometric, Small-World, and Scale-Free graphs [11]. Many of the graphs that model problems arising from everyday life and society belong to the category of Scale-Free networks. The most important characteristic of these graphs is the distinction of a few entities as very important/powerful, while many others are non-important/weak. A characteristic example is the structure of Instagram,

where celebrity accounts have a very large number of followers, while the rest of the population, being less well-known, have significantly fewer. This structure of Scale-Free graphs is called hierarchical.

A major goal in the field of Social Network Analysis is the identification of small groups whose entities communicate with each other more frequently than with entities outside their group. This process is known as Community Detection and already offers applications in our everyday lives. In particular, through this process it becomes possible to recommend movies to users of platforms such as Netflix or songs on Spotify.

1.1 Scope of the Thesis

The evolution of the field of Machine Learning produces increasingly more complex models, which require ever larger volumes of data for their training. At the same time, however, there is also a significant increase in the required training time of these models. This phenomenon is known as the "Curse of Dimensionality" [12]. Consequently, there arises a need to stabilize this training time, a problem that the process of data embedding seeks to solve. This process takes data in their original dimension and embeds them into a lower-dimensional space (latent space), so that they become much easier for models to manage. The goal is for the resulting information to be as "compressed" as possible while at the same time preserving as much information as possible. This process also includes graph data. Therefore, graph data embedding is by no means uncharted territory for the field of Machine Learning. There are various algorithms that implement such embeddings. In particular, at the node level, the algorithms DeepWalk [13] and Node2Vec [14] are among the most widespread. Correspondingly, at the graph level, Deep Neural Networks, either with Attention Mechanisms or with Convolutional Layers, provide information-rich representations.

The field of Social Network Analysis makes extensive use of graph data for the modeling and analysis of real networks. One of the most important goals is the identification of small communication groups among entities, which are called Communities. Nodes with common characteristics or nodes that communicate very frequently are placed together in the same Community. It is now well known that this process becomes more effective when the graphs have first been embedded into spaces characterized by Hyperbolic Geometry, because then the similarity between nodes is translated into distance in Hyperbolic Space. There are various algorithms that undertake this embedding. One such method, and the one that will be used in this thesis, is HyperMap.

As mentioned earlier, embedding graph data for their exploitation by Machine Learning models is a widely used process. In contrast, the use of Machine Learning techniques for the generation of richer embeddings has not received the same degree of attention. Therefore, the main idea of this thesis is the analysis of the latter process and the investigation of its performance. Specifically, Machine Learning models will be used to embed synthetic graph data into Hyperbolic Spaces, with the aim of approximating the operation of HyperMap. The experiments will be conducted in two stages. Initially, the first experiments will concern graphs with 100 nodes, and their goal will be the comparison of

the produced embeddings with those of HyperMap, while, from the experiments that will be carried out on synthetic graphs of larger size, the behavior of the performance of the Neural Networks in relation to graph size will be examined.

1.2 Organization of the Thesis

This thesis is organized into 5 chapters: Chapter 2 provides the theoretical background. First, the necessary definitions of Graph Theory will be presented, so that the reader becomes familiar with the required vocabulary, as well as with some examples of graphs. Then, the main categories of Complex Networks are analyzed through metrics from the field of Social Network Analysis. Next, the connection between Hyperbolic Geometry and Graph Geometry is established, with the aim of revealing the structural relationship between the two. This is followed by a review of the main categories of embedding methods and a more detailed analysis of the PSO, E-PSO, and HyperMap methods. The final part is devoted to the Machine Learning techniques that will be used in the remainder of the thesis. Chapter 3 presents the data collection process and the organization of the applications. Chapter 4 presents the execution of the applications for graphs of all sizes. Finally, Chapter 5 provides a review, presents the conclusions, and discusses the future extensions of this thesis.

Part I

Theoretical Section

Chapter 2

Theoretical background

In this chapter, the theoretical background related to the main subject of this thesis is presented in detail. In particular, elements of Graph Theory and Social Network Analysis, Differential Geometry of Curves and Surfaces, and finally Machine Learning will be presented.

2.1 Graph Theory

2.1.1 Basic Definitions of Graph Theory

Definition 2.1. *(Graph) A graph is any ordered pair $G = (V, E)$ where V is a finite set and E is a set of subsets of V, each of which contains two elements of V. The elements of V are called vertices (or nodes) of G, and the elements of E are called edges (or links) of G. For each edge $e = \{v, u\} \in E$, the vertices v and u are called the endpoints of e, and we say that the vertices v and u are adjacent in G.*

Given a graph G, we use the notation $V(G)$ and $E(G)$ for its vertex set and edge set, respectively. We also define $n(G) = |V(G)|$ and $m(G) = |E(G)|$. The quantity $n(G)$ is called the order of the graph G.

Definition 2.2. *(Adjacency Matrix) A graph can be represented with the aid of a matrix. Specifically, the graph G with $V(G) = \{u_1, u_2, \ldots, u_n\}$ can be represented by an $n \times n$ matrix $A = [a_{i,j}]_{(i,j)\in[n]^2}$ where*

$$a_{ij} = \begin{cases} 1 & \{u_i, u_j\} \in E(G) \\ 0 & \{u_i, u_j\} \notin E(G) \end{cases}$$

Any such matrix is called an Adjacency Matrix of G.

Definition 2.3. *(Undirected Graph) Let G be a graph whose edges have no orientation, that is, $\{u, v\} = \{v, u\}$, $\forall\{v, u\} \in E$. Such a graph is called undirected.*

Conversely, a graph G whose edges have orientation, that is, $(u, v) \neq (v, u)$, is called directed.

Definition 2.4. *(Neighborhood) Let G be an undirected graph and let $S \subseteq V(G)$. The neighborhood of S in G is defined as the set*

$$N_G(S) = \{u \in V(G) \setminus S \mid \exists v \in S : \{v, u\} \in E(G)\},$$

that is, the set of all vertices of G that are adjacent to some vertex $v \in S$ and do not belong to S. If $v \in V(G)$, we define $N_G(v) = N_G(\{v\})$. If for some vertex $x \in V(G)$ it holds that $N_G(x) = \emptyset$, then x is called an isolated vertex.

Let G be a directed graph and let $u \in V(G)$. The in-neighborhood of the node u is the set

$$N^-(u) = \{v \in V(G) : (v, u) \in E(G)\}.$$

Similarly, the out-neighborhood of the node u is the set

$$N^+(u) = \{v \in V(G) : (u, v) \in E(G)\}.$$

Definition 2.5. *(Weighted Graph) A graph G is called weighted if a number $w : E \to \mathbb{R}$ is assigned to each of its edges. This number is called the weight of the edge.*

In a weighted graph, the adjacency matrix is replaced by the weight matrix: $W = [w_{ij}]_{(i,j)\in E}$

Remark 1. *If a graph has loops (edges of the form $\{u, u\}$) or if the edge set contains the same element multiple times, then it is called a multigraph.*

Definition 2.6. *(Planar Graph) A planar multigraph is a pair $\Gamma = (V, A)$ of finite sets satisfying the following properties:*

- *Each element u of V is a point in $\mathbb{R}^2$ and is called a vertex of Γ.*
- *Each element e of A is a subset of $\mathbb{R}^2$, homeomorphic to the open interval $(0, 1)$. The points forming the boundary ∂e of an edge e are called the endpoints of e and belong to V.*
- *Distinct edges have empty intersection.*
- *$V \bigcap (\bigcup_{e\in E} e) = \emptyset$.*

Definition 2.7. *(Node Degree) Let G be an undirected graph. The degree of a node $u \in V$ is defined as the quantity: $deg(u) = |N_G(u)|$.*

Similarly, in a directed graph G, the in-degree of a node $u \in V$ is defined as the quantity $deg^{in}(u) = \sum_{j=1}^{N} a_{ji}$ and the out-degree as $deg^{out}(u) = \sum_{j=1}^{N} a_{ij}$

Definition 2.8. *(Degree Matrix) In an undirected graph G, the degree matrix is defined as the diagonal matrix D, where $D_{ii} = |N_G(u_i)|$*

Definition 2.9. *(Laplacian Matrix) The Laplacian matrix of an undirected graph G is defined as $L = D - A$, where D is the degree matrix and A is the adjacency matrix of G.*

The normalized Laplacian matrix of G can also be defined as $\tilde{L} = I_N - D^{-\frac{1}{2}} A D^{-\frac{1}{2}}$, which is positive semidefinite.

Definition 2.10. *(Path) For every $r \geq 1$, the set*

$$P_r = (\{v_1, \ldots, v_r\}, \{\{v_1, v_2\}, \ldots, \{v_{r-1}, v_r\}\})$$

is called a path, and the vertices v_1 and v_r are called its endpoints, while the remaining vertices are called internal vertices. We also call a path with endpoints x and y an (x, y)-path. The length of a path is the number of its edges.

Definition 2.11. *(Distance) Let G be an undirected graph and let $u, v \in V$ be two of its nodes. The distance between the nodes u and v is defined as the length of the shortest path from u to v and is denoted by $d(u, v)$. Moreover, it holds that $d(u, v) = d(v, u)$.*

In the case where G is a directed graph, the distance between u and v is defined as the length of the shortest directed path.

In the case where G is a weighted graph, the distance between u and v is defined as the sum of the weights of the edges belonging to the shortest path between them.

Definition 2.12. *(Cycle) For every $r \geq 3$, the set*

$$C_r = (\{v_1, \ldots, v_r\}, \{\{v_1, v_2\}, \ldots, \{v_{r-1}, v_r\}, \{v_r, v_1\}\})$$

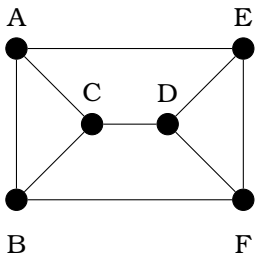
is called a cycle. The length of a cycle is the number of its edges.

Definition 2.13. *(Connectivity) A graph G is called connected if for every pair of vertices $x, y \in V(G)$ there exists an (x, y)-path.*

Definition 2.14. *(Tree) A graph G that contains no cycles is called a forest. If, in addition, it is connected, then it is called a tree. Nodes of degree at most 1 are called leaves of the tree (or of the forest).*

2.1.2 Examples of Graphs

Example 1



Graph G.

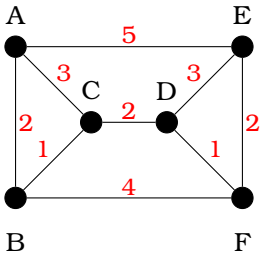
$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 1 & 0 \end{pmatrix}$$

Adjacency matrix of G.

Figure 2.1. *Example of a graph and its adjacency matrix*

Figure 2.1 presents a graph with six nodes, together with its adjacency matrix. The set of nodes is $V = \{A, B, C, D, E, F\}$ and the set of edges is $E(G) = \{\{A, B\}, \{A, C\}, \{B, C\}, \{C, D\}, \{A, E\}, \{B, F\}, \{D, E\}, \{D, F\}, \{E, F\}\}$. The graph is undirected, since A is symmetric, and there are also no weights on the edges. In addition, the degree of each node is 3. In summary, G is a regular, undirected graph, and its edges are unweighted.

Example 2



Weighted graph G.

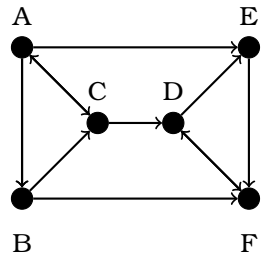
$$W = \begin{pmatrix} 0 & 2 & 3 & 0 & 5 & 0 \\ 2 & 0 & 1 & 0 & 0 & 4 \\ 3 & 1 & 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 & 3 & 1 \\ 5 & 0 & 0 & 3 & 0 & 2 \\ 0 & 4 & 0 & 1 & 2 & 0 \end{pmatrix}$$

Weight matrix of G.

Figure 2.2. *Example of a weighted graph and its weight matrix*

Figure 2.2 presents a graph with six nodes and weighted edges, together with its adjacency matrix. The set of nodes is $V = \{A, B, C, D, E, F\}$ and the set of edges is $E(G) = \{\{A,B\},\{A,C\},\{B,C\},\{C,D\},\{A,E\},\{B,F\},\{D,E\},\{D,F\},\{E,F\}\}$ with corresponding weights $\{2,3,1,2,5,4,3,1,2\}$. The graph is undirected, since A is symmetric, but now the edges carry weights. In addition, the degree of each node is 3. In summary, G is a regular, undirected graph with weighted edges.

Example 3



Directed graph G.

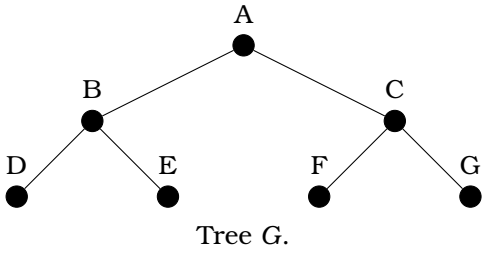
$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{pmatrix}$$

Adjacency matrix of G.

Figure 2.3. *Example of a directed graph and its adjacency matrix*

Figure 2.3 shows a directed graph with 6 vertices. The direction of the edges is visible both in the graph and in the adjacency matrix in Figure (b), since the matrix is not symmetric. As for the degrees of the nodes, for example, we have $deg^{in}(A) = 1$, $deg^{out}(A) = 3$, $deg^{in}(C) = 2$, $deg^{out}(C) = 2$. In summary, G is a regular, directed graph without weights.

Example 4



Tree G.

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 \end{pmatrix}$$

Adjacency matrix of G.

Figure 2.4. *Example of a tree and its adjacency matrix*

Figure 2.4 presents a tree with seven nodes, together with its adjacency matrix. The set of nodes is $V = \{A, B, C, D, E, F, G\}$ and the set of edges is $E(G) = \{\{A, B\}, \{A, C\}, \{B, D\}, \{B, E\}, \{C, F\}, \{C, G\}\}$. Moreover, this graph is a tree since it is connected and contains no cycle. The branching factor is constant and equal to 2 for every node, so this is a binary tree. Its root is node A, while nodes D, E, F, G are the leaves of the tree.

2.2 Complex Network Analysis

The field of Complex Network Analysis [11] aims at modeling real-world social networks and subsequently analyzing them through mathematical tools. The most suitable foundation for this purpose is Graph Theory.

Modeling networks through graphs gives rise to five main classes of networks. In particular, these are Regular, Random, Random Geometric, Scale-Free, and Small-World networks. Although the present thesis focuses mainly on scale-free networks, all of them will be briefly discussed. Regarding the qualitative characteristics of these classes, the degree distribution, clustering coefficient, average path length, and the concept of centrality will be examined.

2.2.1 Metrics for Social Network Analysis

First, the metrics used in the field of Social Network Analysis will be defined, as these will serve as the tools for distinguishing among the five classes of networks.

We begin with the Clustering Coefficient. At a local level, this coefficient gives the probability that two neighbors of a node are connected, while at a global level, it describes the extent to which the nodes of a graph tend to cluster together.

Definition 2.15. *(Local Clustering Coefficient) For each node $u_i \in V(G)$, the Local Clustering Coefficient is defined as:*

$$C_i = \frac{|e_{jk}|}{k_i\,(k_i - 1)} : u_j,\ u_k \in N_G(u_i),\ e_{jk} \in E(G) \tag{2.1}$$

Definition 2.16. *(Global Clustering Coefficient) For an undirected graph G, the Global Clustering Coefficient is defined by the relation:*

$$C = \frac{\sum_{i,j,k} A_{ij} A_{jk} A_{ki}}{\frac{1}{2} \sum_i k_i} \tag{2.2}$$

Centralities are a category of metrics that quantify the "importance" or "influence" of the nodes in a graph. There are several centrality measures, each of which highlights different nodes as "important" depending on how it is computed. The three main ones are the following: Degree Centrality, Betweenness Centrality, and Closeness Centrality.

Definition 2.17. *(Degree Centrality) For each node $u_i \in V(G)$, Degree Centrality is defined as:*

$$C_D(k) = \sum_i a_{ik} \tag{2.3}$$

where a_{ij} is the (i,j)-th entry of the adjacency matrix.

Definition 2.18. *(Betweenness Centrality) For each node $u \in V(G)$, Betweenness Centrality is defined as:*

$$C_B(u) = \sum_{s \neq u \neq t} \frac{\sigma_{st}(u)}{\sigma_{st}} \tag{2.4}$$

where σ_{st} is the number of shortest paths from node s to node t, and $\sigma_{st}(u)$ is the number of shortest paths from node s to node t that pass through u.

Definition 2.19. *(Closeness Centrality) For each node $u \in V(G)$, Closeness Centrality is defined as:*

$$C_C(u) = \frac{1}{\sum_v d(u,v)} \tag{2.5}$$

where $d(u,v)$ is the length of the shortest path between nodes u and v.

Community Detection

In Social Networks, it is observed that nodes tend to form many small communication groups; therefore, in the analysis of such networks, the goal is to identify these groups. This process is called Community Detection or Network Clustering. However, there is no universal definition that fully explains what a community is. The intuitive definition is that nodes which interact very frequently with each other, or nodes that share common characteristics, should belong to the same community.

Illustration 2.1: *Example of clustering a graph into its individual communities. Source: [2]*

Although there are various algorithms for community detection, a common feature of all of them is the extraction of information from the edges connecting the nodes. These algorithms are divided into four categories. First, there are the Node-centric algorithms, which operate on node characteristics. The second category is the Group-centric algorithms, which focus on the connections within communities. Next are the Network-centric algorithms, where communities emerge after successive divisions of the network. Finally, there are the Hierarchy-centric algorithms, which form communities characterized by a hierarchical structure. In every case, metrics from Social Network Analysis are used in order to detect correlations among nodes and thereby identify communities.

2.2.2 Categories of Complex Networks

Regular Networks

The graphs that model Regular Networks are called lattices. In these graphs, each node has the same number of connections, thus yielding a constant degree distribution. At the same time, this crystalline structure also gives rise to the highest clustering coefficient among all classes of Social Networks. Although these networks are compact, they provide the least efficient framework for navigation within them. More specifically, although each node is connected to all neighboring nodes in the lattice, it has no connections to more distant nodes, thereby increasing the average path length. Finally, in these networks, all node centrality measures fail to identify a sufficiently small subset of nodes that could be characterized as "important."

Random Networks

Random Networks are more suitable than Regular Networks for modeling real Social Networks. This is because the connections between nodes are formed randomly, but with the same probability for every independently selected pair, producing a node degree distribution close to the Poisson distribution. The randomness of the connections between nodes reduces the clustering coefficient, thereby giving the network a sparser structure. At the same time, it significantly reduces the average path length, making these networks particularly efficient for navigation. In addition, centrality measures are able to identify a subset of nodes that they characterize as important. It should be noted, however, that

the randomness inherent in these networks leads these measures to disagree on the exact subset.

Random Geometric Networks

Random Geometric Networks are an extremely useful category of networks that can model real Social Networks when the notion of distance affects the relationships between entities. For example, if we want to model the connections of mobile devices in a wireless Wi-Fi network, then connections cannot exist between devices whose distance exceeds the range of the source. This idea is embedded in all the qualitative characteristics of these networks. The distribution of the nodes resembles a Uniform distribution, and the average path length is relatively low, making these networks efficient for navigation within their structure. Finally, the techniques used to identify important nodes tend to result in common sets of nodes being characterized as important. Of course, all of the above results depend on the distance between the nodes, since it is this distance that determines whether a connection between two nodes will exist or not.

To generate such networks, nodes are placed randomly in the unit square ($[0, 1] \times [0, 1]$), and connections between them are established according to a radius R, so that if two nodes are at a distance smaller than the radius, they are connected.

Scale-Free Networks

Scale-Free networks are a limiting case of Random Networks, in which the degree distribution of the nodes follows an Exponential distribution. Their main characteristic is that many nodes have small degree, while very few have extremely large degree. Consequently, these are the networks that can most easily answer the question "Who is the most popular?". In addition, they are the most efficient networks for navigation, since their structure is characterized by the smallest possible clustering coefficient and, at the same time, the smallest average path length. Finally, all centrality measures fully agree in identifying the "important" nodes.

These networks are generated through the rules of Preferential Attachment and Growth. According to the rule of Preferential Attachment, the more connections a node already has, the more likely it is to receive new connections. In addition, according to the rule of Growth, the network expands gradually over time through the addition of new nodes. Overall, in order to generate a scale-free network, the process starts from an empty graph and gradually adds nodes. During the addition of each node, the creation of connections is carried out in such a way that the new node is more likely to connect to nodes that already have many connections.

Small-World Networks

Small-World Networks are an intermediate case between Regular and Random Networks. Their defining characteristic is that most nodes, although not directly connected to each other, can communicate through only a few steps. These networks constitute the

standard model for human relationships and for the spread of viruses in epidemiological models. All of their qualitative characteristics lie, just like the networks themselves, between those of Regular and Random Networks. Therefore, the average path length is relatively small, while the clustering coefficient is relatively high. In addition, the degree distribution of the nodes approaches a Normal distribution, since the nodes have similar degrees. Finally, centrality measures also tend to agree on the "important" nodes, although not to the same extent as in Scale-Free networks.

Their generation is based on the Watts–Strogatz model. This model starts from a regular ring lattice, where each node is connected to its k nearest neighbors. In this graph, some edges are then randomly rewired with probability p. The parameter p controls "how random" the network becomes. Thus, for $p = 0$ a Regular Network is produced, for $p = 1$ a Random Network is produced, while for values $0 < p < 1$ a Small-World Network is produced.

Examples of Complex Networks

In the following diagrams, some examples of graphs for these five categories of networks are presented, along with their node degree distributions.

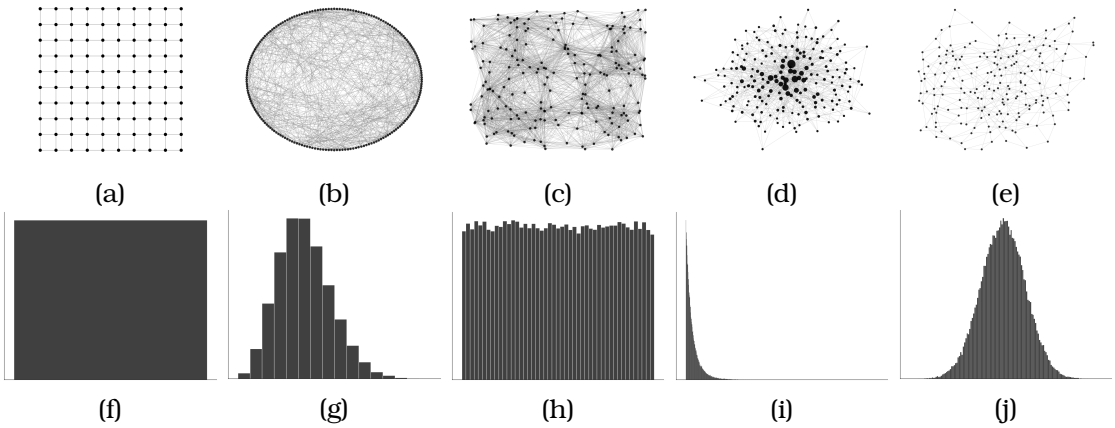


Illustration 2.2: *Examples of complex networks with their node distributions.*

In Figure 2.2, one example from each category of Complex Networks is presented, together with the corresponding node degree distributions. In diagram (a), we observe a Regular network in lattice form. It consists of 100 nodes, and each node has degree 4. Its node degree distribution, shown in diagram (f), is a Dirac delta distribution. Diagrams (b) and (g) concern an example of a Random network with 200 nodes. The node degree distribution approaches a Poisson distribution. Diagrams (c)–(h) arise from an example of a Random Geometric network with 200 nodes. We observe the effect of the connection radius on the formation of links. In regions where the nodes are close to one another, they form tightly connected groups. The node degree distribution for such graphs approaches a uniform distribution. The next two diagrams, (d)–(i), are derived from a Scale-Free network. In particular, in diagram (d), the size of each node is proportional to its degree. In this way, the property of popularity becomes immediately apparent. The node degree distribution for such graphs approaches an exponential distribution. Finally, a

Small-World network is also presented. It is important to observe its intermediate nature between Regular and Random networks. The node degree distribution approaches a normal distribution.

2.3 Hyperbolic Geometry of Graphs

It has been observed that many fields of STEM find solutions to seemingly intractable problems once they recognize the geometry of the space in which the problem exists. Some examples include the analysis of the Theory of Relativity, where gravity is interpreted as a curved geometry [15]. In this section, we will first examine what Hyperbolic Geometry is and why it turns out to be the most suitable setting for certain graphs. Before that, however, some mathematical tools from Differential Geometry that are necessary for this part will be introduced.

2.3.1 Elements of Differential Geometry

In order to distinguish Geometry into its three main categories, the notion of curvature must first be defined. Essentially, curvature is defined as the deviation of a curve from a straight line or the deviation of a surface from a plane. The necessary Mathematical definitions follow [16]:

Elements of Curves

Definition 2.20. *(Parametrized Curve): A parametrized curve in space is a continuous mapping* $a : I \to \mathbb{R}^n$*, where* $I \subseteq \mathbb{R}$ *is an interval.*

Definition 2.21. *(Regular Curve): A curve* $a(t)$ *in space is called regular if and only if it is differentiable at every point of its domain. Specifically, it holds that* $a'(t) \neq 0 \; \forall t \in I$*.*

Definition 2.22. *(Derivative): The derivative of a curve* $a : I \to \mathbb{R}^n$ *is the quantity*

$$a'(t) = (a_1'(t), a_2'(t), \ldots, a_n'(t)) \tag{2.6}$$

This quantity is also called the tangent vector or velocity vector of the curve.

Definition 2.23. *(Speed, Acceleration) The length of this vector is called the speed and is defined as:*

$$u(t) = \|a'(t)\| = \left(a_1'(t)^2 + a_2'(t)^2 + \ldots + a_n'(t)^2\right)^{1/2} \tag{2.7}$$

Similarly, if a' *is also differentiable, then the second derivative*

$$a''(t) = (a_1''(t), a_2''(t), \ldots, a_n''(t)) \tag{2.8}$$

is called the acceleration of a *at the point* t*.*

Definition 2.24. *(Unit-Speed Curve): A curve* $\beta : J \to \mathbb{R}^n$ *with* $\|\beta'(t)\| = 1 \; \forall t \in J$ *is called a unit-speed curve.*

Let $\gamma(t)$, $t \in I$ be a unit-speed curve and let $T(t)$ be the tangent vector of γ at the point $\gamma(t)$. Since the length of $T(t) = \gamma'(t)$ is constantly equal to 1, the derivative $T'(t) = \gamma''(t)$ describes the change in the direction of the vector $T(t)$.

Definition 2.25. *(Curvature): The length of the vector $T'(t)$ is denoted by*

$$k(t) = \|T'(t)\| \tag{2.9}$$

and is called the curvature of γ at $\gamma(t)$.

Elements of Surfaces

Definition 2.26. *(Point Differential) If S_1, S_2 are regular surfaces and $f : S_1 \to S_2$ is a differentiable mapping at $p \in S_1$, we call the (point) differential of f at p (differential of f at p) the mapping*

$$d_p f \equiv T_p f : T_p S_1 \to T_{f(p)} S_2 : w \longmapsto d_p f(w) := (f \circ a)'(0), \tag{2.10}$$

where $a : (-\varepsilon, \varepsilon) \to S_1$ is a differentiable curve with $a(0) = p$ and $a'(0) = w$.

Definition 2.27. *(Gauss Map): Consider an oriented surface S and its orientation $N : S \to \mathbb{R}^3$. Since each $N(p)$ is a unit vector, we may regard N as a mapping of the form*

$$N : S \to S^2 \tag{2.11}$$

The mapping 2.27 is called the Gauss map of S.

Theorem 2.1. *The Gauss map is differentiable.*

Since N is differentiable, its differential is defined as well,

$$d_p N : T_p S \to T_{N(p)} S^2 \tag{2.12}$$

at every point $p \in S$. The tangent space $T_p S$ is the unique 2-dimensional subspace of $\mathbb{R}^3$ that is orthogonal to the vector $N(p)$. On the other hand, by construction, the tangent space $T_{N(p)} S^2$ is also a 2-dimensional subspace of $\mathbb{R}^3$, and it is orthogonal to the position vector (radius) of the point $N(p) \in S^2$. Therefore, the spaces $T_p S$ and $T_{N(p)} S^2$ are two planes parallel to each other, since both are orthogonal to $N(p)$, and they also share the common point 0, hence they coincide, that is,

$$T_p S = T_{N(p)} S^2 \tag{2.13}$$

and thus 2.12 may be regarded as a mapping of the form

$$d_p N : T_p S \to T_p S \tag{2.14}$$

This differential essentially expresses the way in which N varies and therefore, in some sense, describes the "shape" of the surface.

Definition 2.28. *(Negative Map) The negative of the mapping 2.14, that is, the operator*

$$-d_pN : T_pS \to T_pS \tag{2.15}$$

is called the shape operator of S at p.

Let now (U, r, W) be a parametrization of S with $p \in W$. In order to compute the images of $w \in T_pS$ through d_pN, it is sufficient to find the images of the basis vectors $r_u(q)$, $r_v(q)$, where $q := r^{-1}(p)$. For this purpose, we use the curves $a = r \circ \bar{a}$ and $\beta = r \circ \bar{\beta}$, where

$$\bar{a}(t) = (u_0 + t, v_0), \quad \bar{\beta}(t) = (u_0, v_0 + t)$$

respectively, with $(u_0, v_0) := q$. Therefore, from Definition 2.26 of the differential, we have:

$$\begin{aligned} d_pN(r_u(q)) &= d_pN(a'(0)) = (N \circ a)'(0) \\ &= (N \circ r \circ \bar{a})'(0) = \\ &= [D(N \circ r)(q)]\,(\bar{a}'(0)) = [D(N \circ r)(q)]\,(e_1) \\ &= \left.\frac{\partial(N \circ r)}{\partial u}\right|_q \end{aligned}$$

Similarly, we find that

$$d_pN(r_v(q)) = \left.\frac{\partial(N \circ r)}{\partial v}\right|_q$$

Symbolically, we set

$$\boxed{N_u(q) := \left.\frac{\partial(N \circ r)}{\partial u}\right|_q, \quad N_v(q) := \left.\frac{\partial(N \circ r)}{\partial v}\right|_q} \tag{2.16}$$

and hence, finally,

$$\boxed{d_pN(r_u(q)) = N_u(q), \quad d_pN(r_v(q)) = N_v(q)} \tag{2.17}$$

Theorem 2.2. *The shape operator is self-adjoint (or symmetric), that is, for any vectors $w_1, w_2 \in T_pS$, the relation*

$$\langle -d_pN(w_1), w_2 \rangle = \langle w_1, -d_pN(w_2) \rangle$$

holds.

According to the general theory of self-adjoint operators, there exists an orthonormal basis $\{w_1, w_2\}$ of T_pS such that

$$-d_pN(w_1) = k_1 w_1, \quad -d_pN(w_2) = k_2 w_2 \tag{2.18}$$

with $k_1 \geq k_2$. Clearly, w_1, w_2 are eigenvectors of $-d_pN$ and k_1, k_2 are the corresponding eigenvalues. Therefore, the matrix of $-d_pN$ with respect to the orthonormal basis $\{w_1, w_2\}$ is

$$\begin{pmatrix} k_1 & 0 \\ 0 & k_2 \end{pmatrix} \tag{2.19}$$

Definition 2.29. *(Gaussian Curvature) The eigenvalues k_1, k_2 are called the principal curvatures of S at p, while the directions determined by the eigenvectors w_1, w_2 are called the principal directions of S at p. In particular, the Gaussian curvature of S at p is defined as the quantity*

$$K(p) := k_1 \, k_2 \tag{2.20}$$

2.3.2 Distinction Between Categories of Geometry

The distinction between the various categories of Geometry is based on the sign of the Gaussian curvature 2.29. If a surface has zero curvature, then it belongs to the category of Euclidean Geometries and is called flat. Next, surfaces with positive curvature belong to the category of Elliptic Geometries, while those with negative curvature belong to the category of Hyperbolic Geometries.

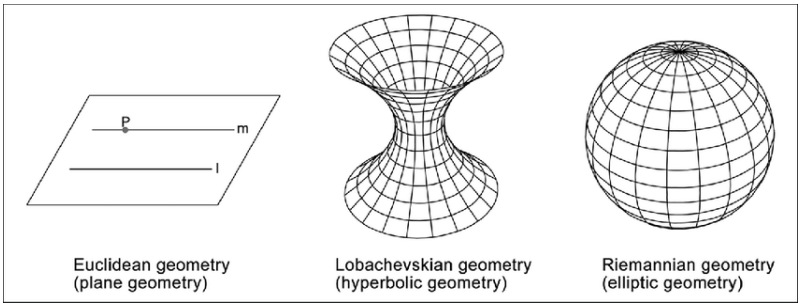


Illustration 2.3: *Schematic representation of three planes. From left to right, a Euclidean, a Hyperbolic, and an Elliptic plane are shown. Source: [3]*

Beyond the different curvature defined on each surface, there is also another qualitative difference based on Euclid's Parallel Postulate. In Euclidean Geometry, given a line and a point outside it, the postulate states that at most one line passes through the point and is parallel to the first. In Hyperbolic Geometries, this is not the case. In Hyperbolic Geometry, for a given line and a point outside it, there exist infinitely many lines through the point that are parallel to the first.

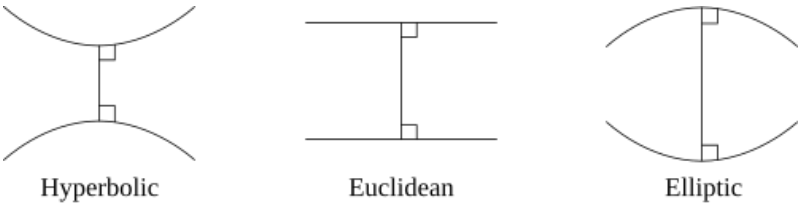


Illustration 2.4: *The Parallel Postulate for the three geometries. Source: [4]*

These differences are summarized in the following table.

Table 2.1. *Distinction among the categories of Geometry.*

Property	Euclidean	Elliptic	Hyperbolic
Curvature K	0	> 0	< 0
Parallel Lines	1	0	∞

2.3.3 Poincaré Model

There are different, equivalent models of Hyperbolic Spaces, each of which emphasizes different properties of these spaces. A very widespread model, and the one that will serve as a basic tool for this thesis, is the Poincaré model. In this model, the entire two-dimensional infinite Hyperbolic Surface $\mathbb{H}^2$ is represented inside a Euclidean disk of radius 1. This hyperbolic surface has constant negative curvature equal to -1. The boundary of the disk is a circle ($\mathbb{S}^1$), consisting of points that do not belong to the hyperbolic surface and are infinitely far away. This boundary is called the boundary at infinity and is denoted by $\partial\mathbb{H}^2$. Finally, in this model, the geodesic lines that define the distance between two points are diameters of circles and circular arcs of Euclidean circles that intersect the boundary orthogonally.

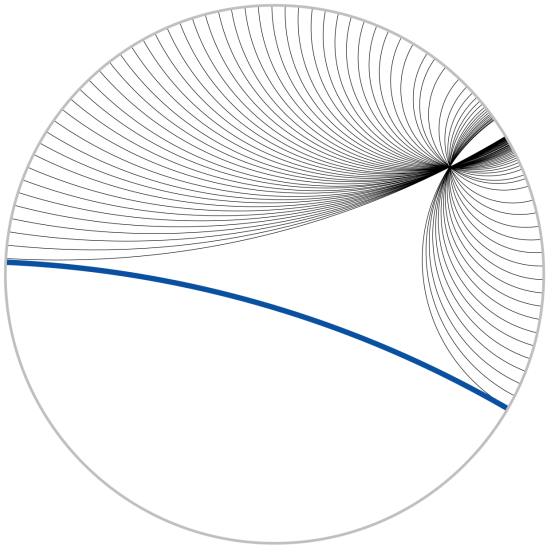


Illustration 2.5: *Schematic representation of the Hyperbolic Surface $\mathbb{H}^2$. For a given line (blue color), the lines parallel to it (black color) are also shown. Source: [5]*

2.3.4 Similarity Between Hyperbolic Geometry and Graph Geometry

First, the definition of isometric embedding will be presented, as it will be needed later.

Definition 2.30. *If $f : M \to N$ is a mapping from a metric space M to another metric space N, then f is an isometric embedding if for every pair of points $x, y \in M$ it holds that:*

$$d_N(f(x), f(y)) = d_M(x, y) \tag{2.21}$$

Hyperbolic Geometry is the most suitable geometry for Graph Geometry [17]. The

major difference between Euclidean and Hyperbolic Geometry is that the latter is "larger." If one considers a disk on a surface with zero curvature, that is, in Euclidean Geometry, it is well known that the area of this disk is given as a function of its radius by the relation:

$$A(r) = \pi r^2 \tag{2.22}$$

Thus, the area of a disk grows quadratically with respect to its radius. Similarly, the circumference of the disk is given by:

$$L(r) = 2\,\pi r \tag{2.23}$$

In contrast, on the surface $\mathbb{H}^2$ with curvature ζ, the area of the disk is given by the relation:

$$A(r) = 2\,\pi\left[\sinh(\zeta r) - 1\right] \tag{2.24}$$

and this quantity grows at the rate of $e^{\zeta r}$. Similarly, the same growth rate also holds for the circumference of the disk, since it is given by the relation:

$$L(r) = 2\,\pi \sinh(\zeta r) \tag{2.25}$$

To finally show that this Geometry is also the one that characterizes Graph Geometry, we now analyze the latter in order to make the similarities apparent. Let us consider trees with branching factor b, the so-called b-ary trees. In this model, the area of a disk of radius r is defined as the number of nodes whose distance from the root is exactly r steps, and the number of nodes contained in this disk is given by the relation:

$$A(r) = \left[(b+1)b^r - 2\right]/(b-1) \tag{2.26}$$

Similarly, the circumference of the disk is defined as the number of nodes that are at distance at most r from the root. Consequently, the circumference is given by the relation:

$$L(r) = (b+1)b^{r-1} \tag{2.27}$$

Both quantities grow exponentially with respect to r.

On the one hand, there is the Hyperbolic Space $\mathbb{H}^2_\zeta$, and on the other hand, the space of b-ary trees. It is evident that, for these structures, the circumference and the area of the disk grow at the rate of $e^{\zeta r}$ if we take $\zeta = \ln(b)$. In this way, isometric embeddings arise, see 2.30, from the space of trees into Hyperbolic Space. Such a process cannot be realized in a Euclidean geometric space, since exponentially more "space" is required there.

2.3.5 Representation of Scale-Free Graphs through Hyperbolic Geometry

The analysis of Scale-Free graphs aims at grouping nodes into categories according to their similarity. For example, for a social media user who has already created some connections, such as friends on Facebook, there arises the need to recommend new

profiles in order to increase their connections. Similarly, in recommendation systems, where a user has watched some movies and is looking for the next one, a grouping of the movie space must be carried out so that a new movie can be successfully recommended. This ability of grouping is fully expressed by spaces with Hyperbolic Geometry.

The problem of grouping nodes is also called classification. In general terms, this means that nodes can be divided into sets, each of which can in turn be subdivided into further subsets, and so on. This separation is approximated by b-ary trees that preserve this hierarchical structure. As discussed in the previous subsection, anything that is approximated by b-adic trees has negative curvature, since it is described by a Hyperbolic Geometry.

Within these hierarchical structures, the distance between nodes is an approximation of the similarity between them. Thus, there arises the need to construct a way in which node similarity is translated into distance in Hyperbolic Space. The process that performs this translation is the following:

Let us consider a node from a graph and represent it as a point in the Euclidean plane $\mathbb{R}^2$. Centered at this point, we consider a Euclidean disk containing the set of features of this node. Therefore, the greater the overlap of two such disks in the Euclidean plane, the greater the similarity between the two nodes. This overlap, however, is also transferred to Hyperbolic Space as a distance, in such a way that the more similar two nodes are, the smaller their hyperbolic distance is. Suppose that the two disks in the Euclidean plane have radii r, r'. If there exists a constant C such that:

$$\frac{1}{C} \leq \frac{r}{r'} \leq C \tag{2.28}$$

and the Euclidean distance between their centers is bounded by the quantity Cr, then there exists a constant C' in Hyperbolic Space that bounds the hyperbolic distance between the nodes, depending only on C. That is, the hyperbolic distance between the nodes depends neither on their position in the Euclidean plane nor on the radii of the disks. Consequently, the distances between nodes, according to their features, can be mapped to distances in Hyperbolic Space.

2.4 Distinction Between Embedding Methods

The research community, recognizing the importance of embedding graphs in hyperbolic spaces, has developed various embedding methods. Each of these relies on different tools to achieve its goal; however, according to Jiang Hao, Lixia Li, Yuanyuan Zeng, Jiajun Fan, and Lijuan Shen [18], the main categories are the following three: Manifold Learning Methods, Likelihood Methods, and Hybrid Methods.

2.4.1 Manifold Learning Methods

The first category consists of manifold learning methods. These rely on data-driven approaches that aim at the Laplacian factorization of the adjacency matrix, exploiting the spectral information of the graph. Specifically, the eigenvalues and eigenvectors of

the Laplacian matrix (or its variants) are computed, with the goal of extracting a lower-dimensional representation of the graph. In this way, the embedding of the graph into a hyperbolic space is achieved, which is particularly suitable for modeling hierarchical and complex topologies, such as those that often appear in real networks.

This spectral approach provides excellent embedding efficiency, since the process is relatively simple and computationally efficient. However, it presents disadvantages with respect to embedding performance, as it does not always take into account the nonlinear and local structural properties of the graph with the same accuracy as more complex methods.

Methods belonging to this category include LaBNE [19] and ncMCE [20].

2.4.2 Likelihood Methods

The second category includes models that achieve embedding by maximizing a likelihood function. The coordinates of the nodes are selected, rather than derived, according to the function that must be maximized. In this category, high embedding performance is achieved, but the complexity of these methods is also very high. Consequently, these methods become impractical for large volumes of data, since their execution time increases dramatically.

Methods in this category include Mercator [21] and HyperMap [22], which will be discussed in more detail later.

2.4.3 Hybrid Methods

The last category is a combination of the previous two, aiming to preserve high embedding accuracy while reducing, as much as possible, their computational complexity.

Some such methods are Hydra [23], L-Hydra [24], MpDHE [18], and Hgena [25].

2.5 Popularity versus Similarity Optimization Method

The Popularity versus Similarity Optimization (PSO) method is a procedure for generating synthetic graphs, developed by Fragkiskos Papadopoulos, Maksim Kitsak, M. Ángeles Serrano, Marián Boguñá, and Dmitri Krioukov [26]. The central idea of this method is based on the property of Homophily [27, 28]. Specifically, similar nodes tend to connect to each other if they are not popular. This property appears very frequently in social networks. For example, an old profile in a social network has a greater probability of attracting connections and, consequently, becoming popular, than a new one.

2.5.1 Parameter Analysis

The PSO method takes four parameters, namely $\beta \in (0, 1]$, $m > 0$, $T \geq 0$, and $\zeta > 0$. Their analysis will be presented below.

Parameter β

The parameter β defines the exponent of the Gamma distribution followed by the degree distribution of the nodes in the graph according to the equation:

$$\gamma = 1 + 1/\beta \geq 2 \tag{2.29}$$

Parameter m

The parameter m is the average number of pre-existing nodes to which a new node will connect. Consequently, this parameter also controls the average degree of the nodes through the relation

$$\bar{k} = 2\,m \tag{2.30}$$

Parameter T

The parameter T, called temperature, controls the average clustering coefficient, $\bar{c}$, of the graph, as presented in [29].

Parameter ζ

The last parameter, ζ, is defined through the curvature coefficient of the Hyperbolic Surface on which the method is carried out. More specifically, it is defined by the relation:

$$\zeta = \sqrt{-K} \tag{2.31}$$

where K is the curvature coefficient. This last parameter does not affect the graph generation process, and is therefore characterized as a "dummy" parameter and is arbitrarily set equal to 1.

2.5.2 Algorithm of the PSO Method

Having defined all the parameters of the method, PSO generates a Scale-Free graph up to t nodes according to the following definition.

The graph generation algorithm of PSO

(1) Initially, the graph is empty.

(2) Assignment and update of coordinates:

(a) At time $i = 1, 2, \ldots, t$, a new node is added to the Hyperbolic Surface with polar coordinates (r_i, θ_i), where the radial coordinate is defined as $r_i = \frac{2}{\zeta} \ln(i)$ and the angular coordinate θ_i is drawn randomly from the uniform distribution on $[0, 2\pi]$

(b) Each existing node $j = 1, 2, \ldots, i-1$ moves by increasing its radial coordinate as follows: $r_j(i) = \beta r_j + (1-\beta) r_i$

(3) Edge construction: Node i connects to each existing node $j = 1, 2, \ldots, i-1$ with a different probability $p_{ij} \equiv p(x_{ij})$ given by the relation:

$$p(x_{ij}) = \frac{1}{1 + e^{\frac{\zeta}{2T}(x_{ij} - R_i)}} \tag{2.32}$$

In the last expression, the hyperbolic distance is defined as:

$$x_{ij} = \frac{1}{\zeta} \, arccosh(cosh(\zeta r_i) \, cosh(\zeta r_j) - sinh(\zeta r_i) \, sinh(\zeta r_j) \, cos(\theta_{ij})), \tag{2.33}$$

where $\theta_{ij} = \pi - |\pi - |\theta_i - \theta_j||$.

The variable R_i arises from the condition that the expected number of nodes to which node i should connect is m. Thus, the following relation holds:

$$R_i = r_i - \frac{2}{\zeta} \ln\left(\frac{2T}{sin(T\pi)} \frac{I_i}{m}\right) \tag{2.34}$$

and

$$I_i = \frac{1}{1-\beta} \left(1 - i^{(1-\beta)}\right) \tag{2.35}$$

2.5.3 Generalized Popularity versus Similarity Optimization Method

The connections between new nodes and old nodes are called external connections. In real networks, new connections are observed at a certain rate, either between old and new nodes or only among old nodes. The PSO method can be extended to handle these connections by adding one more step.

Algorithm of the Generalized PSO Method

The extension of the algorithm to generalized PSO

(4) At each time i, choose a random pair of non-connected nodes $k, l < i$ and connect them with probability

$$p(x_{kl}) = \frac{1}{1 + e^{\frac{\zeta}{2T}(x_{kl} - R_i)}} \tag{2.36}$$

repeating this process until $L > 0$ internal connections have been formed.

Extending the PSO method to its generalized form, the average node degree is now defined as:

$$\bar{k} = 2\,(m + L) \tag{2.37}$$

The additional parameter L defines the rate at which internal links appear, while the parameter m defines the rate of appearance of external links.

2.5.4 External Popularity versus Similarity Optimization Method

The HyperMap method, which will be discussed later, aims to implement the inverse process of the generalized PSO. That is, given a graph, HyperMap attempts to embed it in a Hyperbolic Space in such a way that the embedding approximates the generalized PSO. Specifically, angular and radial coordinates are sought for each node so as to maximize the probability that the embedded graph could have been generated by the generalized PSO method.

However, two problems arise in this attempt. First, having only a static snapshot of the graph, it is impossible to distinguish internal links from external links of the nodes, and second, it is not possible to know the order in which the nodes appeared. The need to overcome these two problems led to a variation of the PSO method, namely E-PSO.

Analysis of External and Internal Links

The E-PSO method is similar to the generalized PSO but uses only external links. Thus, in this setting, instead of each node connecting at the time of its birth to $m = \frac{\bar{k}}{2}$ nodes, it now connects to

$$\bar{m}_i(t) = m + \bar{L}_i(t) \tag{2.38}$$

The parameter $\bar{L}_i(t)$ is the expected number of links that node i will form at time t with the existing nodes $j < i$, and it is calculated by the relation:

$$\bar{L}_i(t) = \int_i^t \tilde{\Pi}(i,j,t)dj \approx \frac{2L(1-\beta)}{(1 - t^{-(1-\beta)})^2(2\beta - 1)} \times \Big[\big(\frac{t}{i}\big)^{2\beta-1} - 1\Big]\big[1 - i^{-(1-\beta)}\big] \tag{2.39}$$

where

$$\tilde{\Pi}(i,j,t) = \int_i^t \Pi(i,j,l)dl \approx \frac{2L(1-\beta)^2}{(1 - t^{-(1-\beta)})^2(2\beta - 1)}(ij)^{-\beta}(t^{2\beta-1} - i^{2\beta-1}) \tag{2.40}$$

and

$$\Pi(i,j,l) = 2L\frac{e^{-\frac{\zeta}{2}(r_i(l)+r_j(l))}}{\int_1^l\int_1^l e^{-\frac{\zeta}{2}(r_i(l)+r_j(l))}didj} = 2L\frac{l^{2\beta-2}(ij)^{-\beta}}{I_l^2} \tag{2.41}$$

In summary, the E-PSO method consists of five parameters, L, m, β, T, ζ, and constructs a new scale-free graph up to t nodes, following exactly the computations of the simple PSO method, with the only difference that the quantity R_i is computed as follows:

$$R_i = r_i - \frac{2}{\zeta}\ln\Big(\frac{2T}{sin(T\pi)}\frac{I_i}{\bar{m}_i(t)}\Big) \tag{2.42}$$

2.6 HyperMap Method

The HyperMap method is the main graph embedding method into Hyperbolic Spaces considered in this thesis. It attempts to embed every graph given to it into a hyperbolic space by maximizing the probability that the embedded graph was generated by the E-PSO method. In order to achieve this, HyperMap produces embeddings by optimizing the distances between nodes in the Hyperbolic Space.

2.6.1 Independence of Coordinates

In the analysis of the PSO method, it was mentioned that nodes are assigned angular coordinates randomly from a Uniform distribution. Therefore, the density function for the angular coordinates is:

$$\rho(\theta) = \frac{1}{2\pi} \tag{2.43}$$

According to the literature, the density function for the radial coordinates is given by:

$$f_t(r) = \frac{\zeta}{2\beta}e^{\frac{\zeta}{2\beta}(r-r_t)} = \frac{\zeta(\gamma-1)}{2}e^{\frac{\zeta(\gamma-1)}{2}(r-r_t)} \tag{2.44}$$

where $r_t = \frac{2}{\zeta}\log(t)$

The radial and angular coordinates in the E-PSO method are independent variables. Therefore, given $\rho(\theta)$ and $f_t(r)$, we obtain their joint density function:

$$\text{Prob}(\{r_i(t),\theta_i\}|\gamma,\zeta) = \frac{1}{(2\pi)^t}\prod_{i=1}^{t} f_t(r_i(t)) \tag{2.45}$$

2.6.2 Local and Global Connection Probabilities

Assume that a graph has grown to t nodes using the E-PSO method. Then the global connection probability $\tilde{p}(x(t))$ is the probability that any two nodes at hyperbolic distance $x(t)$ are connected, and it is defined as:

$$\tilde{p}(x(t)) = \frac{1}{t - i_{\min} + 1} \sum_{i=i_{\min}}^{t} \frac{1}{1 + e^{\frac{\zeta}{2T}(x(t) - R_t + \Delta_i(t))}} \approx \frac{1}{1 + e^{\frac{\zeta}{2T}(x(t) - R_t)}} \tag{2.46}$$

where $i_{\min} = \max\left(2, te^{\frac{\zeta x(t)}{4(1-\beta)}}\right)$, $\Delta_i(t) = \frac{2}{\zeta}\log\left((\frac{t}{i})^{2\beta-1}\frac{m I_i}{\bar{m}_i(t) I_t}\right)$, the variable R_t is given by relation 2.42, $\bar{m}_i(t)$ is given by relation 2.38, and I_i by 2.35.

The $\tilde{p}(x(t))$ in relation 2.46 is called global because it is computed over all pairs of nodes that at time t are at distance $x(t)$. In contrast, the $p(x_{ij})$ in relation 2.32 is called local because it is computed for a specific pair i, j whose hyperbolic distance is x_{ij} at the time of birth of node i.

2.6.3 Local and Global Likelihood Functions

Assume a graph that has grown by the E-PSO method to t nodes according to the parameters m, L, γ, T, ζ, and adjacency matrix A with $a_{ij} \in A$. Then the likelihood function $\mathcal{L}_1 = \mathcal{L}\Big(\{r_i(t), \theta_i\} \mid a_{ij}, m, L, \gamma, T, \zeta\Big)$ is defined. This expresses the probability that node i is assigned coordinates $\{r_i(t), \theta_i\}$, given a_{ij} and m, L, γ, T, ζ. Using Bayes' rule, the function $\mathcal{L}_1$ is transformed into:

$$\mathcal{L}_1 = \frac{\text{Prob}(\{r_i(t), \theta_i\}|\gamma, \zeta)\,\mathcal{L}_2}{\mathcal{L}_3} \tag{2.47}$$

where $\text{Prob}(\{r_i(t), \theta_i\}|\gamma, \zeta)$ is given by relation 2.45. The likelihood function $\mathcal{L}_2 \equiv \mathcal{L}(a_{ij} \mid \{r_i(t), \theta_i\}, m, L, \gamma, T, \zeta)$ gives the probability that the network has adjacency matrix A, given $\{r_i(t), \theta_i\}$ and m, L, γ, T, ζ. It is computed by the relation:

$$\mathcal{L}_2 = \prod_{1 \le j < i \le t} \tilde{p}(x_{ij}(t))^{a_{ij}} \left(1 - \tilde{p}(x_{ij}(t))\right)^{1 - a_{ij}} \tag{2.48}$$

Finally, the likelihood function $\mathcal{L}_3 \equiv \mathcal{L}(a_{ij} \mid m, L, \gamma, T, \zeta)$, which is independent of $\{r_i(t), \theta_i\}$, gives the probability that the E-PSO method with the given parameters has generated the graph with adjacency matrix A.

The likelihood functions in relations 2.47 and 2.48 are called Global Likelihood Functions, since their computation is performed over the entire graph that has grown up to time t.

In contrast to the Global Likelihood Functions, the computation of the Local ones is defined separately for each pair of nodes as the graph grows. From this point on, nodes are named according to their order of appearance in the embedding process. Specifically, each node $i \le t$, at the time of its appearance, is assigned a radial coordinate according to the relation:

$$r_i = \frac{2}{\zeta}\log(i) \tag{2.49}$$

The likelihood function $\mathcal{L}_1^i \equiv \mathcal{L}(\theta_i \mid r_i, \{r_j(i), \theta_j\}, a_{ij}, m, L, \gamma, T, \zeta)_{j<i}$ is defined as the probability that node i is assigned angular coordinate θ_i, given its radial coordinate r_i, the

coordinates of the older nodes $\{r_j(i), \theta_j\}_{j<i}$, and the parameters m, L, γ, T, ζ. Using Bayes' rule, we obtain the relation:

$$\mathcal{L}_1^i = \frac{1}{2\pi} \frac{\mathcal{L}_2^i}{\mathcal{L}_3^i} \tag{2.50}$$

where the function $\mathcal{L}_2^i \equiv \mathcal{L}(a_{ij} \mid r_i, \theta_i, \{r_j(i), \theta_j\}, m, L, \gamma, T, \zeta)_{j<i}$ defines the probability of having the connections $a_{ij}, j < i$ of the adjacency matrix, if the angular coordinate of node i is the value θ_i, given its radial coordinate, the coordinates of the previous nodes, and the model parameters. The likelihood function $\mathcal{L}_3^i \equiv \mathcal{L}(a_{ij} \mid r_i, \{r_j(i), \theta_j\}, m, L, \gamma, T, \zeta)_{j<i}$ is independent of θ_i, and defines the probability that node i has the connections $a_{ij}, j < i$ given these constraints. The computation of $\mathcal{L}_2^i$ is carried out according to the relation:

$$\mathcal{L}_2^i = \prod_{1 \leq j < i} p(x_{ij})^{a_{ij}} \left(1 - p(x_{ij})\right)^{1 - a_{ij}} \tag{2.51}$$

2.6.4 Embedding Procedure

Initially, the Hyperbolic Space is empty, and nodes are gradually added to it. After sorting the nodes in descending order according to their degree, they are assigned their new numbering. The node named $i = 1$ will be the one with the highest degree, $i = 2$ will be the next one, and so on. At each step of the method, $i = 1, 2, \ldots, t$, the node with the corresponding name is inserted into Hyperbolic Space with radial coordinate according to relation 2.49. Then, each of the pre-existing nodes $j = 1, 2, \ldots, i$ receives an increase in its own radial coordinate according to the relation:

$$r_j(i) = \beta\, r_j + (1 - \beta)\, r_i \tag{2.52}$$

so that the notion of popularity is preserved. Then node i is also assigned its angular coordinate, which is obtained by maximizing function 2.51, where the expressions $p(x_{ij})$ are computed according to equation 2.32. Maximizing $\mathcal{L}_2^i$ also preserves the notion of similarity between nodes. An exception to this last procedure is the first node $i = 1$, where at the time of its embedding there are no other nodes in Hyperbolic Space, and so it is assigned a random angular coordinate from the interval $[0, 2\pi)$, selected from the Uniform distribution.

2.6.5 HyperMap Embedding Algorithm

The HyperMap method takes as input a graph, as well as the four parameters T, β, L, m, and performs its embedding according to the following algorithm.

The HyperMap embedding algorithm

1. Sort the node degrees in descending order $k_1 > k_2 > \ldots > k_t$, breaking ties arbitrarily.
2. Name node i, $i = 1, 2, \ldots, t$, as the node with degree k_i.
3. Node $i = 1$ is born and is assigned radial coordinate $r_1 = 0$ and a random angular coordinate $\vartheta_1 \in [0, 2\pi)$.
4. Repeat for $i = 2, 3, \ldots, t$:
   (a) Node i is born and is assigned radial coordinate $r_i = \frac{2}{\zeta} \log(i)$.
   (b) Increase the radial coordinate of each existing node $j < i$ according to the equation $r_j(i) = \beta r_j + (1 - \beta) r_i$.
   (c) Assign to node i the angular coordinate ϑ_i that maximizes the function L_2^i, as defined in equation 2.51.
5. End repetition.

2.7 Machine Learning

For the experiments of this thesis, various Machine Learning models will be used. Since the thesis revolves around graph data, it was considered important that these models be Graph Neural Networks, as they are more suitable for this purpose. More specifically, these are Neural Networks designed to extract the additional information encoded by this type of data. First, the types of Machine Learning will be analyzed, and then Neural Networks will be discussed. Their differences will be highlighted, as well as why the latter are more appropriate in the context of this thesis.

Machine Learning is a relatively new field in Computer Science, albeit one that has developed rapidly, a fact that is due to the tremendous increase in available data [30], and at the same time to the development of computational machines capable of supporting computations on them. Feeding Machine Learning models with large amounts of data enables them to discover patterns and approach the solution of difficult and complex problems. In conclusion, it is a field that offers solutions to problems for which an algorithmic approach is not straightforward.

2.7.1 Types of Machine Learning

Supervised Learning

The first type of Machine Learning is Supervised Learning. In this setting, the data used are labeled. This means that if, for example, there is image data, it is also known what each image depicts, and this information can be used to guide the models during training. In this type of learning, there are two possible objectives. The first objective is Classification. In Classification, the goal is to categorize data into distinct classes

according to their labels. A characteristic example is the classification of images into dogs and cats. The second objective is called Regression. The difference between Regression and Classification is that in the former, the output does not consist of discrete categories, i.e., classes, but rather of a continuous range of values. Therefore, the model is not called upon to choose one of the classes, but to produce the output.

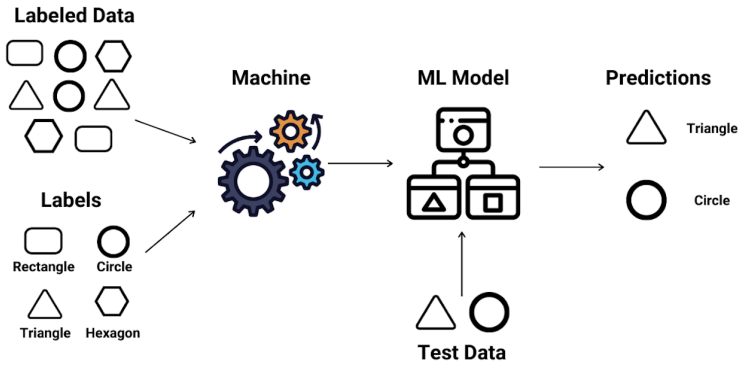


Illustration 2.6: *Schematic illustration of Supervised Learning. Source: [6]*

Unsupervised Learning

The second type is called Unsupervised Learning, and in contrast to the first, the data are unlabeled. In this category, the model is required, without any human intervention, to discover correlations among the data. After the successful discovery of these correlations, the grouping of the data follows. The goal is for the objects within one group to be as similar as possible, while each object should differ as much as possible from the objects in the other groups. This process is called Clustering.

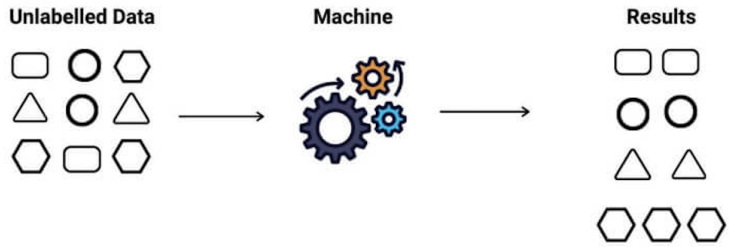


Illustration 2.7: *Schematic illustration of Unsupervised Learning. Source: [7]*

Reinforcement Learning

The final type is called Reinforcement Learning. In this type, the system interacts with the environment and, depending on its actions, receives either a reward or a penalty. Thus, through this process, the model discovers the optimal way to explore the environment. In this way, if the reward-or-penalty process is properly defined (which is the only

human intervention), then the optimal trajectory of the system will also be the solution to the problem at hand.

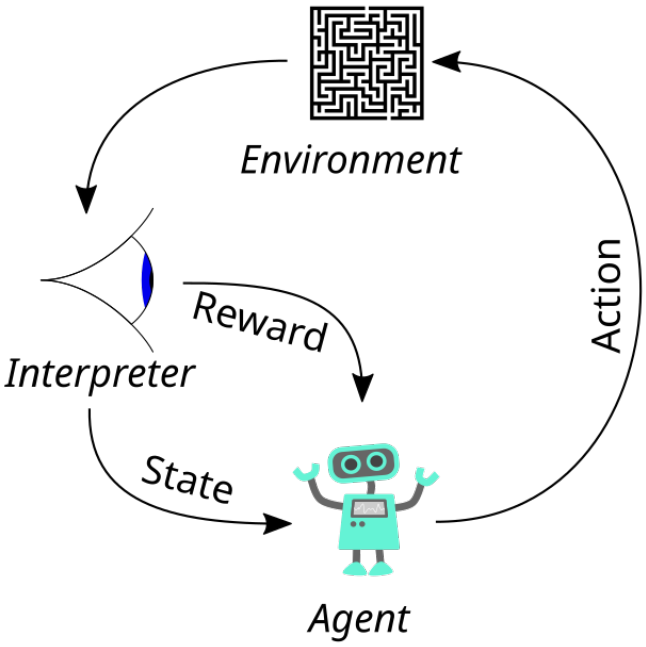


Illustration 2.8: *Schematic illustration of Reinforcement Learning. Source: [7]*

2.7.2 Training

The architecture of each Machine Learning model affects to a very large extent its performance in every execution of the respective model. Architectures with many hidden layers tend to perform better when the data consist of images, since each layer has the ability to focus on different features of the image. On the other hand, when the data are "simpler," for example data represented by vectors, then shallower architectures tend to perform better. In any case, the training of models is the critical stage that allows them to extract information from the data.

The training of Machine Learning models is characterized by a function called the Loss Function. In the case of Supervised Learning, during training, the models receive data and produce an output. The Loss Function computes the deviation of the produced output from the true one. The goal of training is to discover the appropriate weights for the models so that each produced output deviates as little as possible from the true one. This is done through an iterative process, where initially the model weights are random and therefore produce random outputs. The number of iterations is called the number of epochs and, consequently, each iteration is called an epoch. The deviation between the true and the produced output defines a direction for adjusting the weights. The process of defining this direction is performed by the optimizer, since there are various ways in which it can be determined. Along this direction, the learning rate is also defined, which determines the magnitude of the changes applied to the weights. This iterative process leads to weight values that minimize the Loss Function. It is particularly important,

however, that this minimization does not adversely affect the predictive ability of the models, that is, their performance on new data. For this reason, training is usually not continued until the complete minimization of the Loss Function, but stops slightly earlier.

In Unsupervised Learning, the Loss Function still exists, but it does not compute the deviation between the produced and the true output, since there are no labeled data. In this type of learning, the output is more arbitrary and therefore the Loss Function, which is defined through the problem itself, models some qualitative characteristic of the problem. The rest of the process remains the same.

In Reinforcement Learning, the training process consists of repeating the same experiment multiple times in order to discover the optimal solution in the minimum number of steps.

2.7.3 Neural Networks

Neural Networks take their inspiration from the biological neuron. In the human body, a neuron receives an initial stimulus, which either activates it or not, and as a result the signal is transmitted when activation occurs. From there continues the McCulloch–Pitts model, which likens neurons to logical gates that are activated or not if their stimulus (input) exceeds a certain threshold.

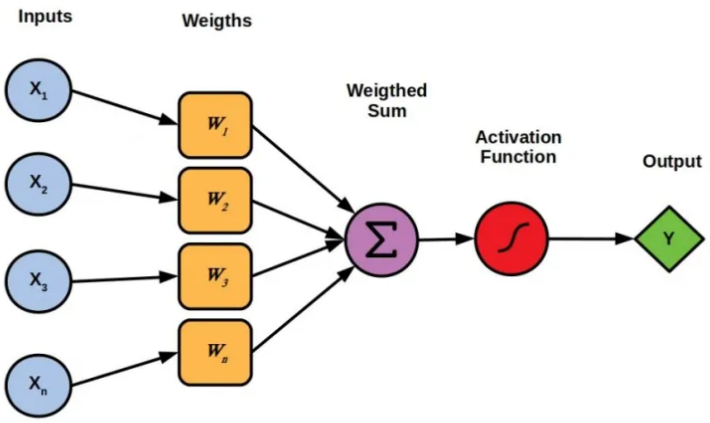


Illustration 2.9: *Schematic illustration of a Perceptron. Source: [8]*

Perceptron Model

The analogy of the McCulloch–Pitts model is represented by the Perceptron 2.9. More specifically, we have a number of inputs, synaptic weights, and a bias, which together create the total excitation. Finally, there is an activation function that takes the total excitation as input and determines whether the neuron will ultimately be activated or not. The mathematical formulation is the following:

The Perceptron Model

- Neuron state $\begin{cases} y = 0 & \text{, inactive neuron} \\ y = 1 & \text{, activated neuron} \end{cases}$
- Inputs: $x_1, x_2, \ldots, x_n$
- Synaptic Weights: $w_1, w_2, \ldots, w_n$
- Bias: w_0
- Total Excitation: $u = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$
- Activation Function: $y = f(u) \in \{0, 1\}$

Matrix Form: $y = f(\tilde{W}^T X)$,
where $\tilde{W}^T = \begin{bmatrix} w_0, & w_1, & w_2, & \cdots, & w_n \end{bmatrix}$ and $X^T = \begin{bmatrix} 1, & x_1, & x_2, & \cdots, & x_n \end{bmatrix}$

It has been defined that the activation function takes values in the set $\{0, 1\}$, but this is not restrictive and depends entirely on the problem. In addition, it is common for the bias to be defined as $w_0 = 1$.

A pattern is called each element of the dataset. The output of the neuron automatically gives the class to which each pattern belongs. Thus, every pattern given as input to the neuron and leading, through the set of computations, to output 0, will be classified in class 0. Correspondingly, the patterns that produce output 1 will belong to class 1. Geometrically, there arises a surface of dimension $n - 1$ (n is the number of features of each pattern) such that, if the patterns are plotted in Euclidean space $\mathbb{R}^n$, those belonging to class 0 lie below this surface, while those of class 1 lie above it.

Multi-Layer Perceptron Model

The linear nature of the Perceptron model, which provides a solution only to linear problems (problems where the separating surface is a plane), is not sufficient for solving every problem. In particular, in many problems there arises the need for a more complex separating surface. This problem is addressed by the Multi-Layer Perceptron model.

The structure of the Multi-Layer Perceptron model consists of an input layer, hidden layers, and an output layer. The input layer consists of n Perceptron neurons, each of which receives all the inputs x_i. According to the way described previously, each neuron belonging to the input layer produces an output that is fed to every neuron of the first hidden layer. Similarly, each neuron of the first hidden layer produces an output that is transferred to every neuron of the second hidden layer, and so on, until the information reaches the output layer. The number of neurons in each hidden layer may vary. For this reason, the notions of width and depth are introduced. Width is the maximum number of neurons among the hidden layers, while depth is the number of hidden layers. Finally, the output layer contains m Perceptron neurons, where the output of each one constitutes one of the outputs. Such a structure is presented in the following diagram.

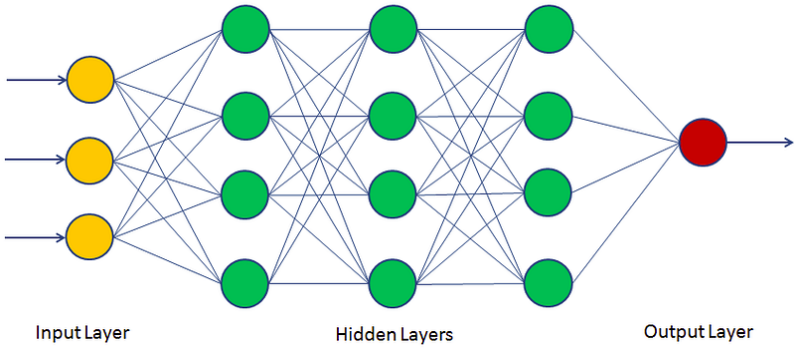


Illustration 2.10: *Example of the architecture of a Multi-Layer Perceptron. Source: [9]*

The structure shown in Figure 2.10 depicts a Multi-Layer Perceptron. It is defined by an input layer with three neurons, which are responsible for processing the information they receive from the data. Then, the output produced by the input layer is transferred to the hidden layers. In the present architecture, there are three hidden layers, each of which has four neurons. After the successive transformation of the produced outputs into inputs for the next layer, the information reaches the output layer. In this specific case, it is characterized by a single neuron.

The process mentioned previously, where information is transferred from layer to layer starting from the input layer and ending at the output layer, is presented through its mathematical definition.

The Multi-Layer Perceptron Model

For a dataset X where $x^{(k)} = (x_1^{(k)}, x_2^{(k)}, \ldots, x_p^{(k)})$, the algorithm of the Multi-Layer Perceptron is the following:

$$h_i^{(1)} = f^{(1)}\left(\sum_j w_{ij}^{(1)} x_j + b_i^{(1)}\right)$$
$$h_i^{(2)} = f^{(2)}\left(\sum_j w_{ij}^{(2)} h_j^{(1)} + b_i^{(2)}\right)$$
$$\vdots$$
$$h_i^{(n-1)} = f^{(n-1)}\left(\sum_j w_{ij}^{(n-1)} h_j^{(n-2)} + b_i^{(n-1)}\right)$$
$$y_i = f^{(n)}\left(\sum_j w_{ij}^{(n)} h_j^{(n-1)} + b_i^{(n)}\right)$$

Here, $f^{(l)}$ denotes the activation function of layer l, since each layer may have a different activation function. In addition, each layer may have its own bias $b^{(l)}$.

In the analysis of the Perceptron and the Multi-Layer Perceptron, activation functions have been mentioned. These are divided into linear and nonlinear ones depending on their form. Some of them are defined below.

- Step function: $\begin{cases} f(x) = -1 & x \leq 0 \\ f(x) = 1 & x > 0 \end{cases}$

- Sigmoid: $f(x) = \sigma(x) = \frac{1}{1+e^{-x}}$
- Hyperbolic Tangent: $f(x) = \tanh(x) = \frac{2}{1+e^{-2x}} - 1$
- ReLU: $f(x) = \max(0, x)$
- Softplus: $f(x) = \log(1 + e^{x})$

Having defined the basic building block, the Perceptron, and its generalization, the MLP, it is now possible to define Neural Networks. Essentially, these are MLPs but with more complex structures. For example, in a Neural Network there may be missing connections between neurons located in adjacent layers, or connections may be added between neurons that are not in adjacent layers. Likewise, choices can be made regarding the existence or absence of bias.

Convolutional Neural Networks

The need to solve increasingly complex problems leads to increasingly larger Neural Network architectures. This growth of Neural Networks, although it makes them extremely capable, creates an important problem. More specifically, as the number of neurons increases, so does the training time of the network and the required amount of training data. Therefore, we must move toward a smarter way of addressing these complex problems. This is precisely what Convolutional Neural Networks (CNNs) provide.

According to [31], Convolutional Neural Networks are Machine Learning models that include a convolution layer. This layer reduces the number of model parameters and consequently addresses the aforementioned problem. The convolution layer operates on data with Euclidean structure. This structure appears, for example, in image data, where the arrangement of pixels is organized into a two-dimensional matrix. The convolutional layer uses filters that move over the entire input matrix and perform convolutions (additions and/or multiplications). In this way, they produce a new matrix, usually of smaller dimension, which has aggregated information about the local features of the image while at the same time focusing on its different regions.

2.7.4 Graph Neural Networks

Graph data, however, do not have Euclidean structure. As a result, a generalization of Convolutional Neural Networks must arise that also includes this type of data. Graph Neural Networks (GNNs) are the evolution of these models. Their goal is to learn a function that combines both the information from the features of each node and the information provided by neighboring nodes, ultimately producing a new representation.

The analysis of the structure of these models, as well as the architectures that will be used in this thesis, will form the next part of the thesis. The aim is to clarify the way in which these models manage to extract information from this kind of data.

The categorization of Graph Neural Networks is divided into four main categories, as also stated in [32]. In particular, we have Graph Convolutional Neural Networks,

Graph Attention Networks, Spatial–Temporal Graph Neural Networks, and some Hybrid methods.

Graph Convolutional Neural Networks

Graph Convolutional Neural Networks (GCNs) implement the convolution operation at the graph level. This means that they attempt to combine the features of a node with the information carried by its neighboring nodes. These models are subdivided into spectral and spatial ones. Since the number of neighbors may vary from node to node, spectral GCNs use the Fourier transform and transform the input information into the frequency domain. In this way they also implement the convolution operation. In this thesis, the GCN model was used, which belongs to the category of spectral models. In particular, for an input signal $x \in \mathbb{R}^d$ and a convolutional filter $g \in \mathbb{R}^d$, the convolution operation is given by the relation:

$$\begin{aligned} x *_G g &= F^{-1}(F(x) \odot F(g)) \\ &= U(U^T x \odot U^T g) \\ &= U \hat{g} U^T x \end{aligned} \tag{2.53}$$

where $*_G$ denotes the convolution operation on graphs, the function $F(x) = U^T x$ denotes the Fourier transform on graphs, with $F^{-1}(F(x)) = U F(x)$ denoting the inverse Fourier transform on graphs, $\odot$ denotes the Hadamard product, and $\hat{g} = diag(U^T g)$.

In contrast, spatial GCNs implement the convolution operation based on the information provided by the neighbors of a node in order to obtain a richer representation. The problem of having a different number of neighbors appears in these models as well. In order to overcome this, sampling techniques are used that define both the number of neighbors and which neighbors will be included in each operation. In the context of this thesis, the GraphSAGE model [33] was used. This model fixes a number of neighbors for each node, while the sampling process determines which neighbors will be included. The convolution operation is performed according to the following relation:

$$X_{u_i}^{(t)} = \sigma\Big(W^{(t)}\, g^{(t)}(X_i^{(t-1)}, \{X_j^{(t-1)}, \forall j \in S_{N(u_i)}\})\Big) \tag{2.54}$$

where $W^{(t)}$ is the trainable weight matrix and $g^{(t)}$ is the aggregation function at layer t, respectively. In addition, σ is a nonlinear activation function and the set $S_{N_G(u_i)}$ is a random sample from the set of neighbors of node u_i.

Graph Attention Networks

Among the set of neighboring nodes, it is expected that some neighbors provide more information than others. The attention mechanism allows the model to categorize neighbors according to the information they provide and to focus on those with the greatest contribution. This is the idea behind Graph Attention Networks. In the context of this thesis, the GAT model [34] was used, which assigns to different neighbors a weight that

indicates the quality of the information they provide. This is done through the relation:

$$X_i^{(t)} = \sigma\Big(\sum_{j \in N(u_i)} a(X_i^{(t-1)}, X_j^{(t-1)}) W^{(t-1)} X_j^{(t-1)} \Big) \tag{2.55}$$

where $a(\cdot)$ is the attention mechanism and $X_i^{(t)}$ is the hidden state of node u_i at layer t.

Spatial–Temporal Graph Neural Networks

Spatial–Temporal Graph Neural Networks are another category of Graph Neural Networks used when the nature of the graphs is dynamic rather than static. More specifically, these models attempt to extract information when the inputs from the data are time-dependent and the connected nodes are interdependent. In that case, the spatial properties of the data must first be extracted, and then their temporal correlation must be sought. The first process is carried out through diffusion convolution, and the second through GRUs (Gated Recurrent Units).

Part II

Experimental Section

Chapter 3

Analysis and Design

In this chapter, the preparation of the applications is presented. Specifically, the process of generating synthetic data will be analyzed, as well as the optimization of the environment for Machine Learning (choice of optimizer, learning rate, number of epochs, and model architecture).

3.1 Data Collection

3.1.1 Barabasi–Albert Synthetic Graph Generation Method

For the following experiments, synthetic Scale-Free graphs were used, generated with the Barabasi–Albert method from the NetworkX library [35]. This process starts from an empty graph and adds nodes one by one until a total of n nodes have been inserted. More specifically, when a node is added, it creates m connections, preferentially attaching to nodes with the highest degree. In this way, the node degree distribution ends up following a Gamma distribution: $f(x;\gamma) = x^{-\gamma}$.

In total, 20 synthetic graphs were created using the above procedure. All of them have 100 nodes, and the edge index takes values from 7 to 33. This means that the graphs become increasingly dense, since each newly added node connects to an increasing number of nodes. For the rest of the thesis, each graph was named according to the value of parameter m.

3.1.2 Parameter Computation

As mentioned previously, the HyperMap method requires, at initialization, the values of the parameters T, β, L, m of the graph it is called to embed. Thus, the next step is to compute these values for each graph.

Temperature T

The temperature T was computed indirectly from the average clustering coefficient of the graph. After computing the clustering coefficient of each node, we then have the ability to define the average clustering coefficient as the mean of the individual coefficients. This metric takes values in the interval $[0, 1]$, and the closer it is to the value 1, the more cohesive the graph under consideration is. However, the temperature parameter

behaves oppositely, since as we move away from the value 0 and approach the value 1, the cohesiveness of the graph decreases. For this reason, we defined the temperature parameter through the linear transformation $f(x) = 1 - x$.

Parameter β

The Gamma distribution that best describes the node degree distribution was found, and its exponent was taken. Thus, according to the literature, the parameter β is obtained from the equation:

$$\beta = \frac{1}{\gamma - 1} \tag{3.1}$$

Parameter m

The parameter m is obtained from historical data during the evolution of the graph. More specifically, this parameter is defined as the average number of connections of the nodes at the time they are inserted into the graph. Of course, in the case where no such historical data exist, this parameter takes the value of the minimum node degree. Since the graph generation method that was followed does not provide the required historical data, parameter m was defined using the second approach.

Parameter L

Finally, the average node degree is computed:

$$\bar{k} = \frac{1}{|G|} \sum_{u \in G} deg(u) \tag{3.2}$$

This is obtained as the mean of the individual degrees. Thus, knowing both parameter m and the average node degree $\bar{k}$, the last parameter L is defined by the relation

$$L = \frac{\bar{k} - 2m}{2} \tag{3.3}$$

The values of the parameters computed as described above are presented in the following table.

Table 3.1. *Statistical properties of the synthetic graphs.*

Graph	Temperature	γ	L	m
07	0.9	3.3	1.5	5
09	0.8	3.6	1.2	7
10	0.8	3.5	4.0	5
12	0.8	4.8	5.6	5
13	0.8	5.5	3.3	8
18	0.7	3.4	5.8	9
19	0.7	4.3	2.4	13
20	0.7	3.9	10.0	6
21	0.7	4.0	4.6	12
23	0.6	3.9	1.7	16
24	0.6	3.6	0.2	18
25	0.6	4.0	9.8	9
26	0.6	3.7	1.2	18
27	0.6	4.0	4.7	15
28	0.6	3.5	1.2	19
29	0.6	3.6	16.6	4
30	0.5	3.7	18.0	3
31	0.6	3.8	7.4	14
32	0.5	3.6	8.8	13
33	0.5	3.8	16.1	6

3.1.3 Distributions of Angular Coordinates and Hyperbolic Distances

In order to evaluate the experimental results, it was necessary to embed each synthetic graph using the HyperMap method. In this way, the angular and radial coordinates of each node were obtained. As mentioned in the definition of the HyperMap method, the computation of the radial coordinates is deterministic, since it depends entirely on the degree of each node. Therefore, it is more important to examine the angular coordinates of the nodes. The distribution of the angular coordinates of each graph, as computed by the HyperMap method, is presented below.

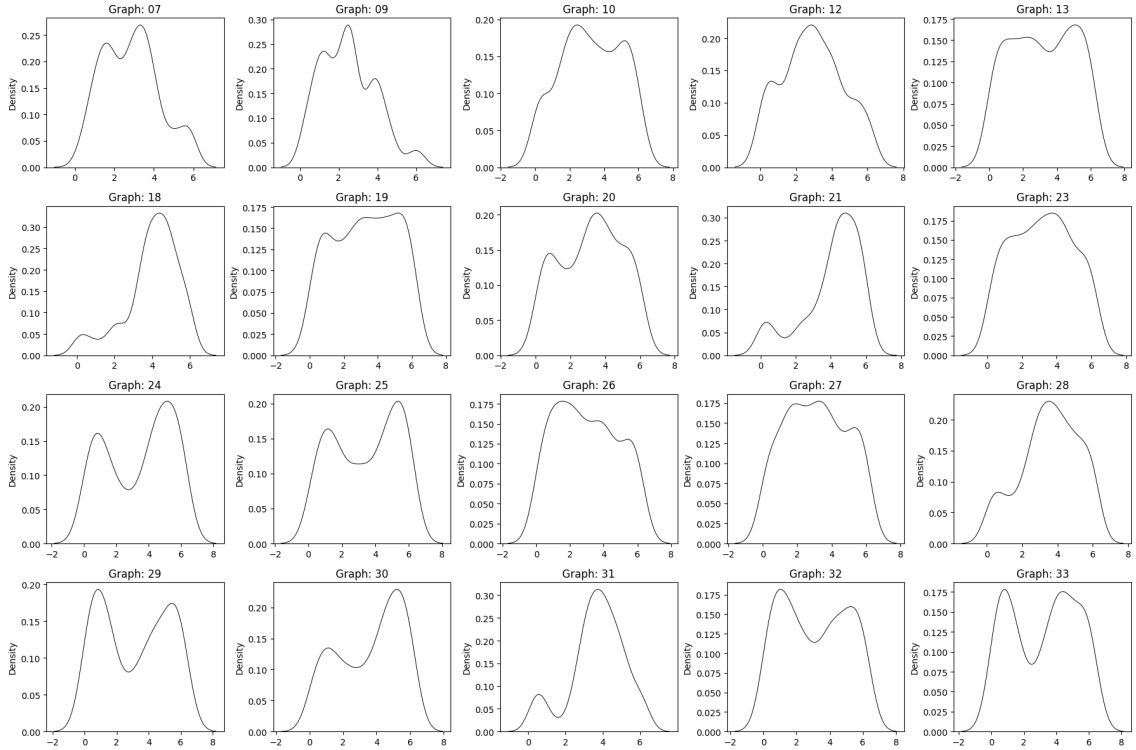


Illustration 3.1: *The distribution of the angular coordinates of the synthetic graphs using the HyperMap method.*

At the same time, in order to approximate the E-PSO method, the HyperMap method, as also stated in the literature, optimizes the distances between nodes in hyperbolic space. Therefore, it is necessary to compare the hyperbolic distances of the applications in this thesis with those of the HyperMap method. In the next figure, the distributions of the hyperbolic distances obtained by the HyperMap method are presented. Each distribution gives the number of node pairs for each value of hyperbolic distance.

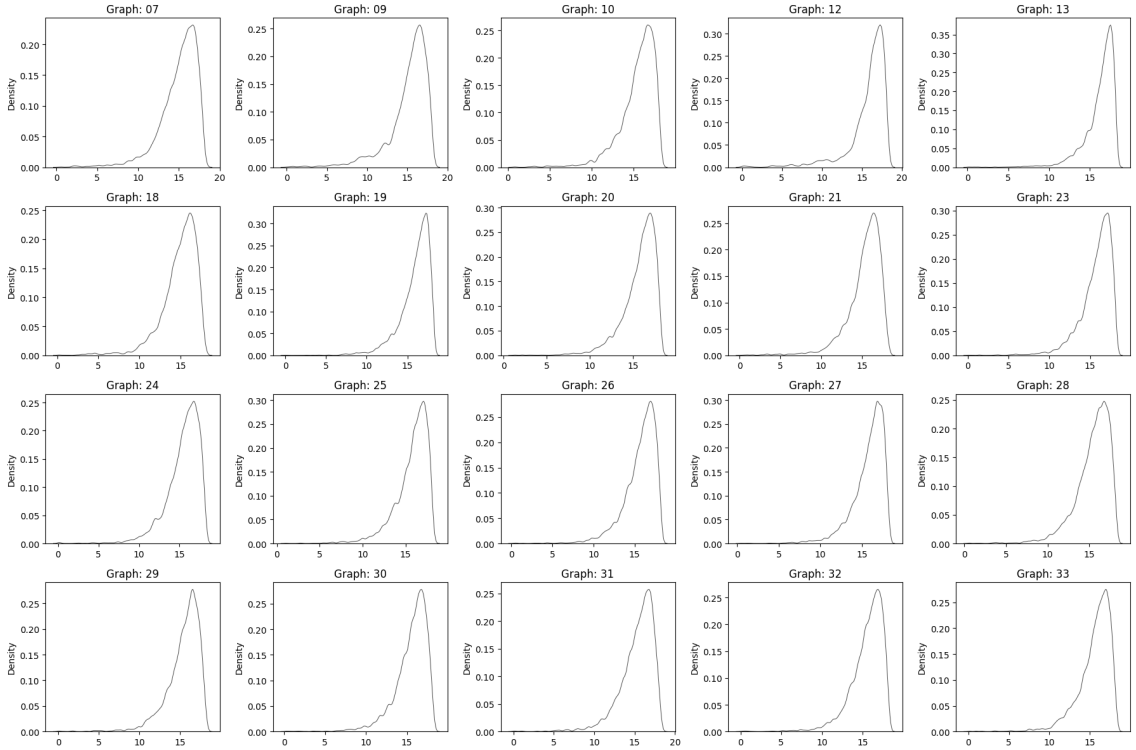


Illustration 3.2: *The distribution of the hyperbolic distances of the synthetic graphs using the HyperMap method.*

3.2 Application Initialization

3.2.1 Optimizer and Number of Epochs

The initial step in defining the framework in which the applications will be carried out is to determine the number of epochs as well as the optimizer. Thus, it was decided to embed one graph using a specific model for different numbers of epochs and different optimizers, in order to choose the optimal combination.

The evaluation of each experiment is based on the Kullback–Leibler divergence and the Wasserstein distance between the distributions of the angular coordinates obtained by the HyperMap method and those produced by the model.

The first graph was selected for this procedure, together with the model GAT_16_4. The optimizers under consideration are Adam and Adagrad, and the numbers of epochs are 10, 20, 30, 40, 50, and 60.

The results of this procedure are presented in the following table.

Table 3.2. *Results of the experimental analysis for the different optimizers as well as the different values of the number of epochs.*

Optimizer	# Epochs	KL Divergence	Wasserstein Distance
Adam	10	1.389	0.005
	20	1.633	0.003
	30	1.638	0.003
	40	1.757	0.002
	50	1.830	0.002
	60	1.841	0.002
Adagrad	10	1.484	0.004
	20	1.424	0.005
	30	1.481	0.005
	40	1.485	0.005
	50	1.491	0.005
	60	1.504	0.005

According to the literature [36, 37, 38], between KL Divergence and Wasserstein Distance, the latter is more reliable because it also takes into account differences in the support of each distribution, unlike KL Divergence; furthermore, it is symmetric and satisfies the triangle inequality. In contrast, KL Divergence is used to compute the deviation of one distribution from a ground truth distribution. If we consider two distributions P, Q, then the problem of the asymmetry of KL Divergence can be bypassed by computing the following quantity:

$$\text{Symmetrized KL Divergence} = \frac{KL(P\|Q) + KL(Q\|P)}{2} \tag{3.4}$$

where $KL(P\|Q) = \sum_{x \in X} P(x) \log \frac{P(x)}{Q(x)}$.
Correspondingly, the Wasserstein distance is defined as

$$W_2(P\|Q) = \inf_{\pi}\left(\frac{1}{n}\sum_{i=1}^{n} \|P(i) - Q(\pi(i))\|^{1/2}\right) \tag{3.5}$$

that is, the infimum over all permutations π.
Therefore, taking the above into account, as well as the increase in computational complexity as the number of training epochs increases, the optimal combination is found to be **Adam** and **40 epochs**.

3.2.2 Learning Rate

Next, different values for the learning rate must be tested, so that the optimal one can be selected. For the combination of epochs and optimizer selected previously, the experiment was repeated 4 times with different learning-rate values. The values under consideration are 0.1, 0.01, 0.001, and 0.0001. The Kullback–Leibler divergence and the Wasserstein distance will again be used to quantify the results. These are shown in the following table.

Table 3.3. *Results of the experiments for searching the optimal learning rate.*

Learning Rate	KL Divergence	Wasserstein Distance
0.1	1.819	0.004
0.01	1.756	0.003
0.001	1.938	0.003
0.0001	1.934	0.003

The learning rate affects the rate at which the weights of the Neural Network being trained change. Therefore, the smaller this value becomes, the smaller the changes made to the model weights will be. This is also evident in Table 3.3. The Wasserstein distance decreases, but these changes are negligible. On the other hand, the KL divergence increases as the learning rate decreases, showing that the model does not have enough time to adapt sufficiently quickly to the data. Consequently, the value **0.01** will be selected.

3.2.3 Model Comparison

For the following experiments, 9 machine-learning models will be used. More specifically, these are Graph Neural Networks, that is, models designed for graph-related problems. Their difference lies in the fact that the architecture of each layer is configured so as to collect information about the nodes of the graph. Some models focus on the neighbors of each node, while others focus on the edges of each node and on how information can be extracted from them. In the present case, three types were selected: Graph Convolutional Networks, SAGE, and GAT. For each type, a different number of neurons and hidden layers was used. The models used are the following:

Table 3.4. *The architecture of each model with respect to the number of hidden layers and the number of neurons in each of them.*

Model	Number of Neurons			
	Hidden Layer 1	Hidden Layer 2	Hidden Layer 3	Output Layer
CONV_16_4	16	4	-	1
CONV_16_8	16	8	-	
CONV_16_4_4	16	4	4	
CONV_16_8_4	16	8	4	
CONV_32_16_8	32	16	8	
SAGE_16_4	16	4	-	
SAGE_16_8	16	8	-	
GAT_16_4	16	4	-	
GAT_16_8	16	8	-	

The goal of the experiment at this stage is to predict the angular coordinates of each node through training and to approximate the HyperMap method.

In the definition of the Machine Learning models, 9 models were defined, each with different architectures. These differences appear in the different numbers of layers, different numbers of neurons per layer, and even in the different types of convolutional layers. This part of the analysis aims to determine whether there is a significant difference among the models in their predictive ability, and, if so, to identify the optimal model.

Thus, an iterative procedure was carried out for each architecture, in which each model was trained and produced angular coordinates for three graphs. More specifically, graphs 9, 10, and 12 were used for this experiment. In this way, conclusions were drawn in order to determine whether any model is systematically better than the others. As before, the quality of each embedding will be judged based on the Kullback–Leibler divergence and the Wasserstein distance.

The results are shown in the following table.

Table 3.5. *Performance of the models on synthetic graphs 9, 10, and 12.*

Architecture	Graph	KL Divergence	Wasserstein Distance	Avg. KL	Avg. W
CONV_16_4	9	2.110	0.005	1.758	0.0046
	10	1.603	0.003		
	12	1.562	0.006		
CONV_16_8	9	1.988	0.005	1.708	0.0046
	10	1.512	0.004		
	12	1.625	0.005		
CONV_16_4_4	9	1.978	0.005	1.875	0.0046
	10	1.914	0.005		
	12	1.733	0.004		
CONV_16_8_4	9	1.857	0.005	1.736	0.0055
	10	1.534	0.003		
	12	1.818	0.004		
CONV_32_16_8	9	1.777	0.004	1.766	0.004
	10	1.786	0.004		
	12	1.737	0.004		
SAGE_16_4	9	2.053	0.003	1.918	0.004
	10	2.042	0.003		
	12	1.662	0.006		
SAGE_16_8	9	1.646	0.004	1.839	0.003
	10	2.027	0.002		
	12	1.845	0.003		
GAT_16_4	9	2.057	0.007	1.725	0.0053
	10	1.470	0.004		
	12	1.650	0.005		
GAT_16_8	9	2.682	0.004	2.152	0.0053
	10	2.168	0.007		
	12	1.607	0.005		

It appears that all architectures present similar results, having extremely small values in Wasserstein distance and good values for Kullback–Leibler divergence. Three models can be distinguished as better: CONV_16_8, CONV_16_8_4, and GAT_16_4, which are characterized by the smallest values in KL divergence. In order to better distinguish whether any model outperforms the others, the distribution of angular coordinates as well as the distribution of hyperbolic distances of the models will be presented using a new synthetic graph, namely graph 13.

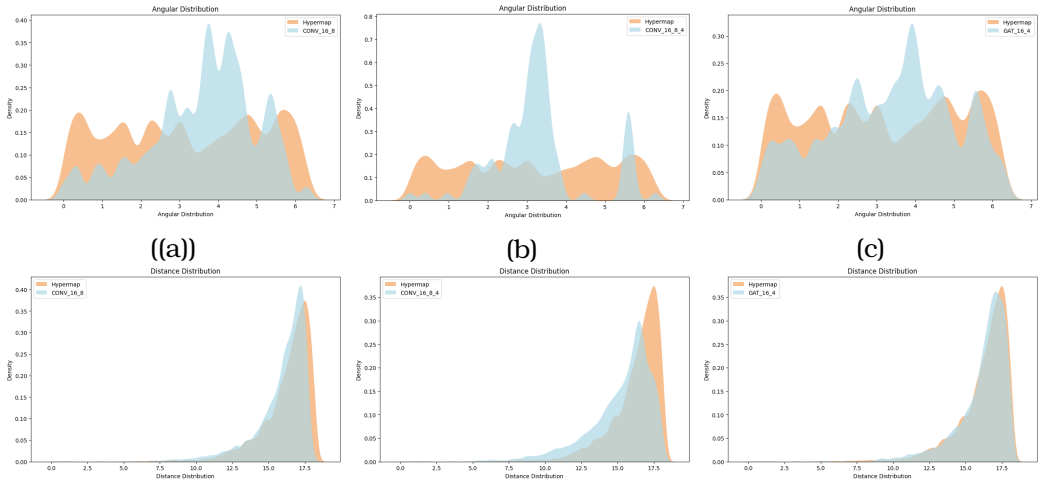


Illustration 3.3: *The distributions of the angular coordinates and hyperbolic distances produced by the models CONV_16_8, CONV_16_8_4, and GAT_16_4.*

Figures (a), (b), and (c) concern the distributions of the angular coordinates of the models CONV_16_8, CONV_16_8_4, and GAT_16_4. Correspondingly, figures (d), (e), and (f) present the distributions of the hyperbolic distances between nodes. In each figure, the distribution produced by the HyperMap method is shown in orange, while the distribution produced by the trained model is shown in blue. It becomes evident that the GAT_16_4 model (see (c) and (f)) approximates the distribution of angular coordinates better than the other two, while at the same time it approximates almost perfectly the distribution of hyperbolic distances.

3.2.4 Loss Function

The Loss Function is, as also discussed in the theoretical framework of Machine Learning, the part of the model that guides the training stage. The changes in the model weights are directed by this function. Consequently, the choice of an appropriate Loss Function is of utmost importance for the performance of any model.

The Loss Function that was selected is inspired by the HyperMap method in order to approximate its embedding process. More specifically, the HyperMap method bases the quality of the embedding it produces entirely on the appropriate choice of angular coordinates. This choice is made through the maximization of function 2.51. In constructing the Loss Function for the Machine Learning models, the Negative Log-Likelihood method was used, transforming L_2^i into

$$-\log\left(\mathcal{L}_2^i\right) = -\sum_{1\leq j<i}\left[a_{ij}\log p(x_{ij}) + (1-a_{ij})\log\left(1-p(x_{ij})\right)\right] \tag{3.6}$$

so that it becomes a decreasing function. In this way, the process of producing angular coordinates by the models follows the embedding process of the HyperMap method.

Chapter 4

Detailed Results

In this chapter, the experiments conducted within the framework of this thesis will be analyzed. In the previous section, various experiments were carried out in order to select the optimal conditions for the subsequent experiments. Specifically, the Adam optimizer with 40 epochs was selected, the learning rate was set to 0.01, and GAT_16_4 was chosen as the architecture. In this section, the coordinates of the remaining graphs in Hyperbolic Space will be sought and evaluated through Kullback–Leibler divergence as well as through the diagrams concerning the distributions of the angular coordinates and the hyperbolic distances.

4.1 Experiments on Graphs of Size 100

The GAT_16_4 model was trained on graphs 18, 19, 20, 21, 23, 24, 25, 26, 27, and 28 in order to obtain the coordinates of the nodes in Hyperbolic Space. The next diagram presents the Kullback–Leibler divergences for the distributions of the angular coordinates.

4.1.1 Analysis of the Angular Coordinates

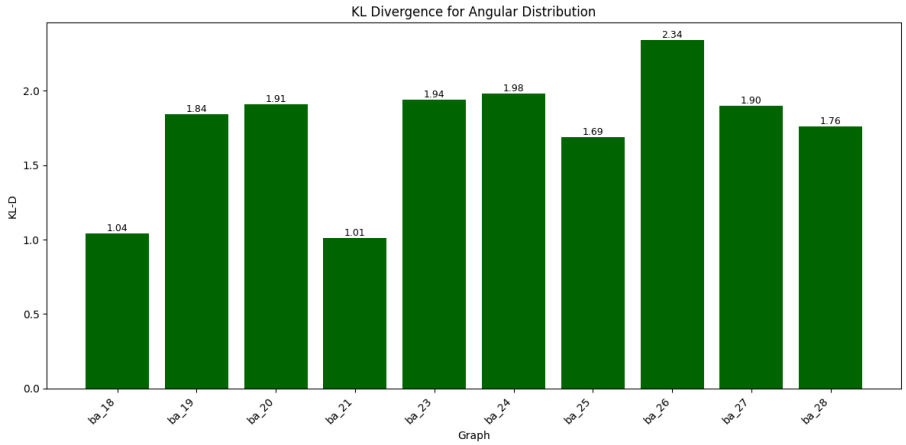


Illustration 4.1: *Kullback-Leibler divergence for the distributions of the angular coordinates.*

The analysis will begin with the Kullback–Leibler divergences for the angular-coordinate distributions. As mentioned in the Theoretical Background, the HyperMap method does

not operate by optimizing angular coordinates, but rather by minimizing hyperbolic distances. Moreover, during the first step of the method, node 1 is assigned a random angular coordinate from a Uniform distribution on the interval $[0, 2\pi)$. Consequently, repeating the embedding of a graph produces different angular coordinates for each node. The element that remains stable across all repetitions is the qualitative characteristics of the embedding. More specifically, these are the distances between the nodes and the Communities that are formed. Through the KL divergence of the angular-coordinate distributions, it is evident that the distributions are not very far from each other, but a further analysis is required.

In Figure 4.2, the distributions of the angular coordinates are presented in comparison with those of the HyperMap method.

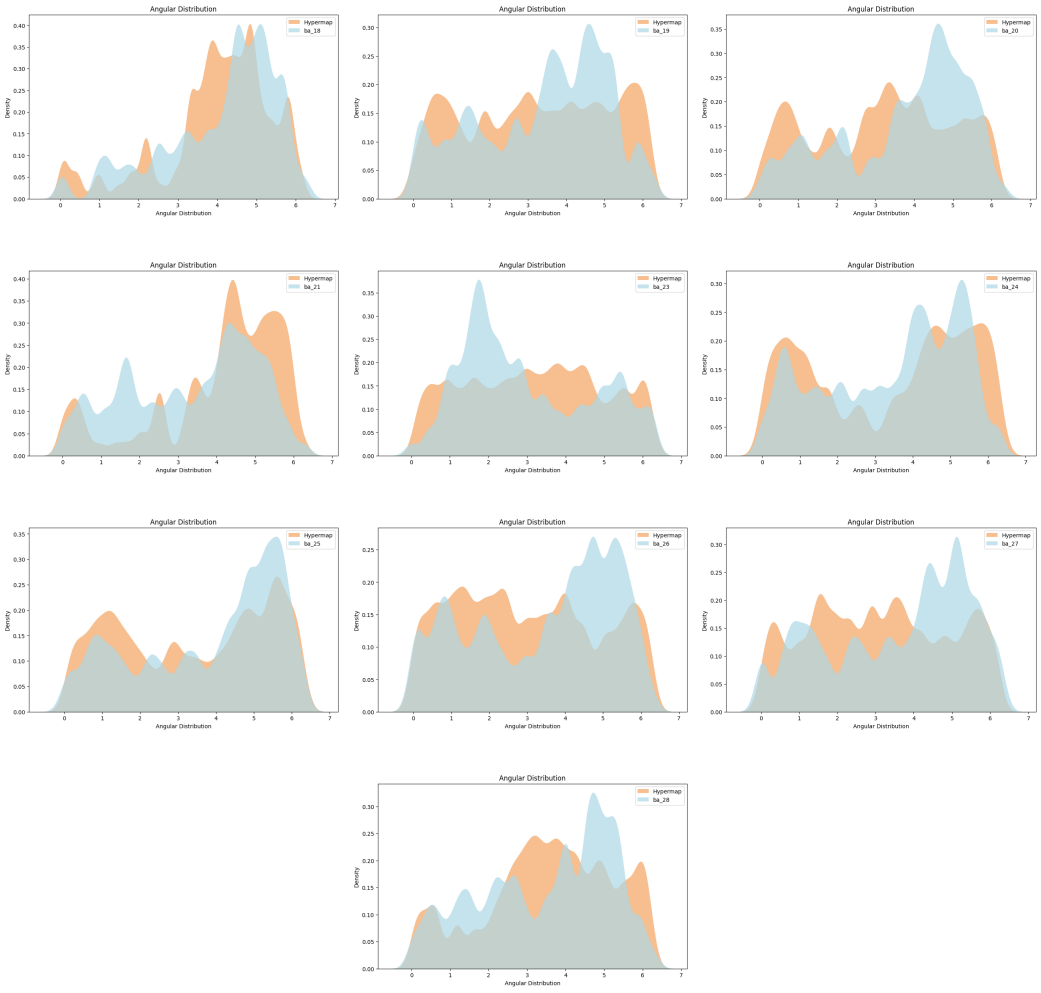


Illustration 4.2: *The distributions of the angular coordinates for the experiments on graphs of size 100.*

Analyzing the above diagrams, the conclusion arises that the model manages to reproduce almost the same qualitative characteristics as the HyperMap method. The qualitative characteristics that need to be preserved are both the number of peaks and the distances between the peaks within the same distribution. The relative positions between the peaks

of different distributions (that is, the one produced by the model and the one produced by HyperMap) are not important due to the observation noted above. The number of peaks reflects the number of Communities in the graph. Therefore, by observing the diagrams, a consistency between the model and HyperMap becomes apparent, indicating that both methods recognize the same number of Communities. In addition, the distances between the peaks translate into hyperbolic distances between the Communities in Hyperbolic Space. Thus, the farther apart two peaks are, the more confident the method is in the distinction between the Communities, since it recognizes a much lower similarity among the corresponding nodes. Here, there seems to be a slight differentiation between the methods for some graphs. In the graphs where the HyperMap method clearly identifies distinct Communities and strongly distinguishes nodes within a Community from those outside it, the model seems to agree and reproduce them (see (a), (d), (f), (g)). In contrast, in the graphs where HyperMap does not clearly identify distinct Communities, the model differs and forms its own Communities. This is visible in the diagrams where HyperMap distributes the nodes through a distribution that approaches the Uniform one and therefore does not exhibit high peaks, whereas the model creates its own peaks (see (b), (e), (h), (i)).

4.1.2 Analysis of the Hyperbolic Distances

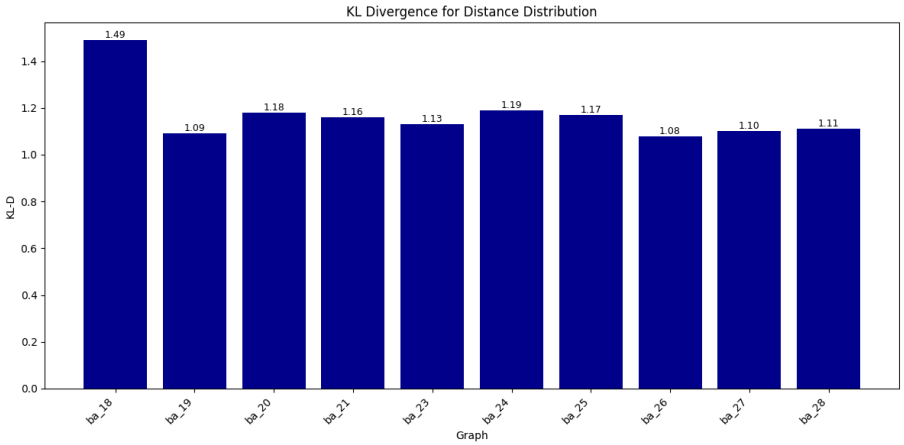


Illustration 4.3: *Kullback-Leibler divergence for the distributions of the hyperbolic distances.*

Moving on to the distribution of the hyperbolic distances, the analysis will again begin with the Kullback–Leibler divergences. We recall that the naming of the graphs refers to the edge-creation parameter in the Barabasi–Albert method. As this index increases, no clear trend emerges in the performance of the model. More specifically, from graph 19, which has the second smallest index and is therefore the second sparsest graph in the set, to graph 28, which is the densest, the divergences fluctuate around approximately the same values. The exception is graph 18, which is the sparsest one, where indeed a larger divergence appears. Therefore, it can be claimed that the performance of the model does not depend on the density of the graph.

In the following diagrams, the distributions of the hyperbolic distances produced by the model are presented in contrast with those of the HyperMap method for each graph.

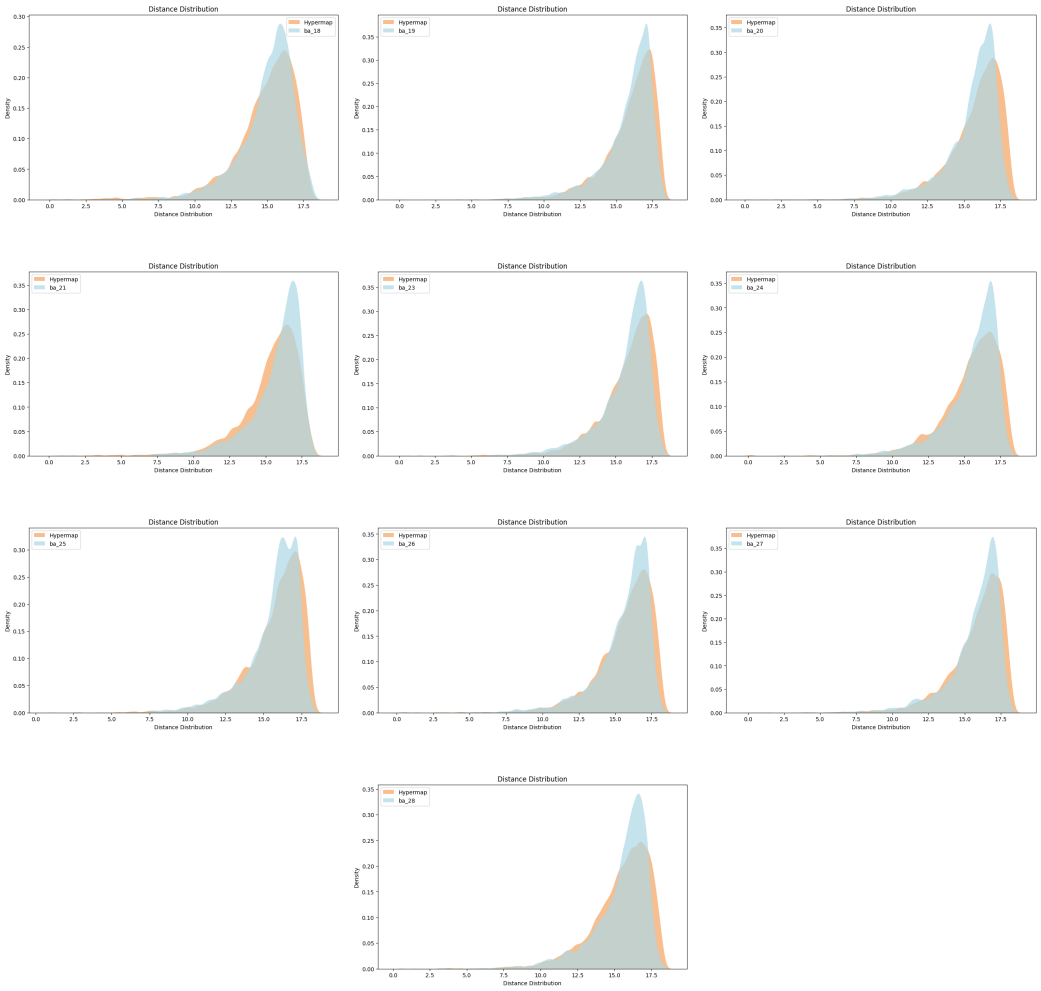


Illustration 4.4: *The distributions of the hyperbolic distances for the experiments on graphs of size 100.*

In each of the above diagrams, the distribution of the hyperbolic distances produced by the model (blue distribution) is presented in contrast to that of the HyperMap method (orange distribution). Initially, a strong correlation between the distributions is observed, which is encouraging, since it indicates that the embeddings produced by the model are close to those of the HyperMap method. The feature that stands out in every diagram is that the distributions produced by the model exhibit higher peaks on the right-hand side. This means that the model produces embeddings in which most node pairs are characterized by the largest hyperbolic distances. This observation is nevertheless discouraging for the model's performance, since the objective is the minimization of these distances.

The minimization of hyperbolic distances is not merely an approach that happens to characterize the HyperMap method. It is a fundamental characteristic of embeddings in Hyperbolic Spaces for highlighting the property of Similarity. Consequently, by producing larger hyperbolic distances, the model also expresses an inability to recognize similarities

between nodes in comparison with the HyperMap method.

4.2 Experiments on Larger Graphs

All the previous experiments were conducted on graph data with 100 nodes. In order to investigate the performance of the GAT_16_4 model as a function of graph size, four new graphs were created with 100, 200, 300, and 400 nodes. The training of the model on these graphs will be repeated, and the existence or non-existence of a performance trend will be examined.

4.2.1 Analysis of the Angular Coordinates

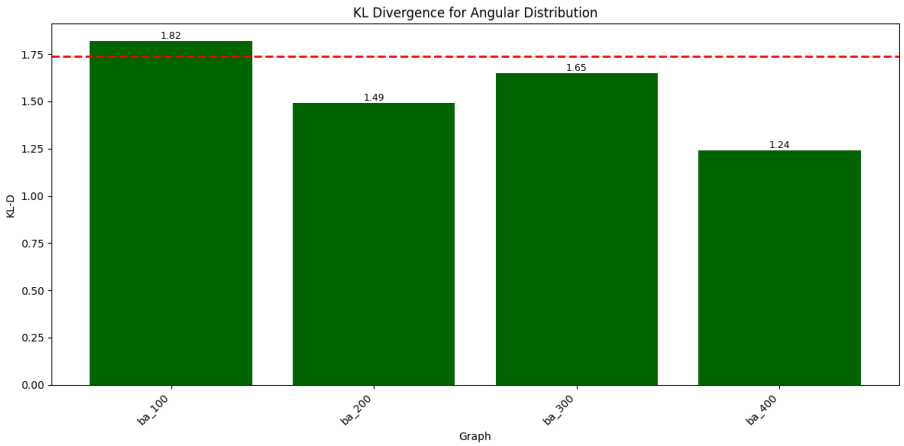


Illustration 4.5: *Kullback-Leibler divergence for the distributions of the angular coordinates. The red dashed line shows the mean divergence for the graphs of size 100.*

The above diagram presents the Kullback–Leibler divergences for the angular-coordinate distributions from the embeddings produced by the model for the larger graphs. Initially, no clear trend in the model's performance that depends on graph size can be detected. The divergences show a decreasing tendency, but this tendency does not appear to be stable. This becomes apparent when comparing the divergences of the graphs of size 200, 300, and 400 with that of the graph of size 100, as well as with the mean of the divergences of the previous distributions 4.1. The fact that the divergence of the distributions is consistently lower highlights a greater agreement between the angular-coordinate distributions of HyperMap and those of the model. Consequently, this agreement is also transferred to the separation of the Communities. Of course, for this analysis, the angular-coordinate distributions for each embedding must also be presented.

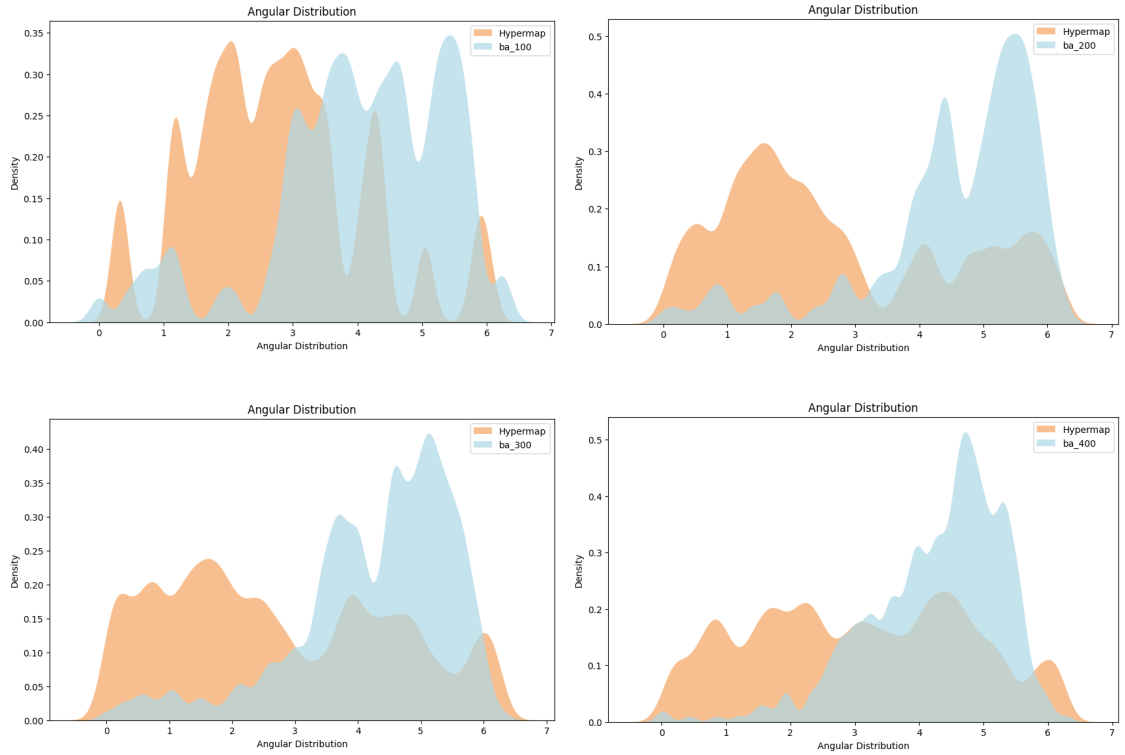


Illustration 4.6: *The distributions of the angular coordinates for the experiments on the larger graphs.*

The observation that prevailed in the analysis of the angular-coordinate distributions for graphs of size 100 also holds for the larger graphs. When the HyperMap method identifies Communities, that is, high peaks in the angular-coordinate distribution diagram, then the model follows suit. In contrast, when the HyperMap method does not identify distinct Communities, the model diverges and highlights some of its own. For the present embeddings, the model and the method agree in the first two cases (see (a) and (b)), while they diverge in the other two (see (c) and (d)). Starting from the embedding of the graph of size 100, there appears to be complete agreement in the number of Communities (7 peaks for both embeddings). In addition, both methods also agree on the relative distances between the Communities. More specifically, four peaks are presented in both methods that are placed relatively close to each other, and another two peaks that are more distant from the others. Moving on to graph 200, the HyperMap method identifies two Communities and places them far apart from one another, while the model also identifies two Communities but ends up placing them at a much smaller distance. The same analysis is appropriate for the embedding of the graph of size 300. Finally, for the embedding of the graph of size 400, the model identifies one Community, in contrast to the HyperMap method, which produces a distribution that approaches the Uniform one.

4.2.2 Analysis of the Hyperbolic Distances

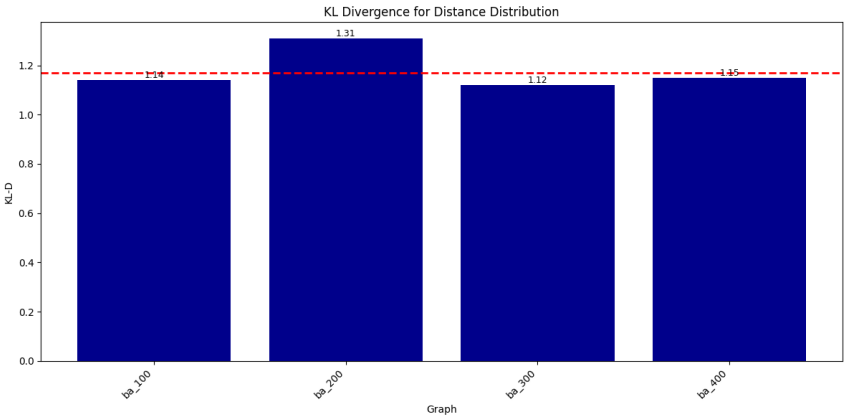


Illustration 4.7: *Kullback-Leibler divergence for the distributions of the hyperbolic distances. The red dashed line refers to the mean divergence of the hyperbolic-distance distributions for the embeddings of the graphs of size 100.*

The analysis concerning the distributions of the hyperbolic distances will again begin with the Kullback–Leibler divergences. The first observation is that all divergences fluctuate close to the mean of the previous embeddings. The only differentiation observed appears in the embedding of graph 200, where a larger divergence occurs. For a more appropriate analysis of all the embeddings, the distributions of the hyperbolic distances must also be presented.

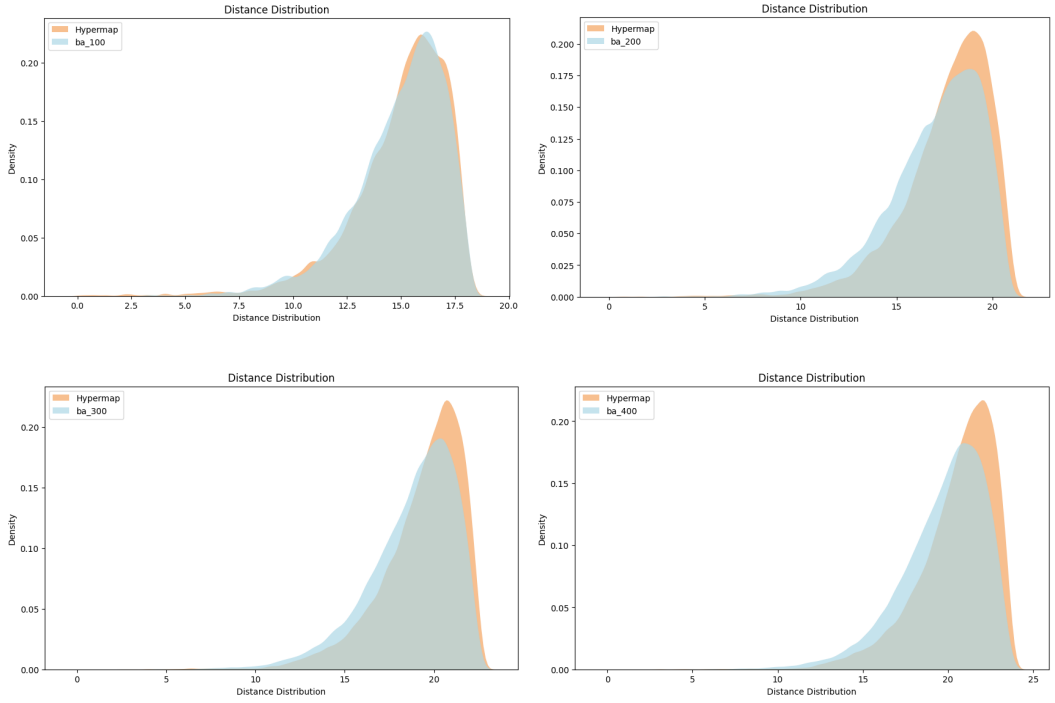


Illustration 4.8: *The distributions of the hyperbolic distances for the experiments on the larger graphs.*

The diagrams concerning the distributions of the hyperbolic distances present a trend in the performance of the model that is related to graph size. Starting from the embedding of the graph of size 100, it is characterized by a close agreement between the two hyperbolic-distance distributions. The model likely produces slightly more pairs with large hyperbolic distances, but this difference is almost imperceptible. As the number of nodes in the graph that the model is called to embed increases, the peak of the distribution decreases. This is particularly important because it reduces the number of node pairs characterized by large hyperbolic distances. In addition, the peaks of the distributions produced by the model (blue color) are slightly shifted to the left. This fact also indicates a reduction in the maximum observed hyperbolic distance among the pairs. Summarizing the above analysis, the reduction in the number of node pairs characterized by large hyperbolic distances is proportional to graph size, while the horizontal shift of the peak, although present, is not proportional to graph size.

Part III

Conclusion

Chapter 5

Conclusion

5.1 Overview

Within the framework of this thesis, the theoretical background for embedding graphs in Hyperbolic Spaces was analyzed. Specifically, the mathematical tools for distinguishing the field of Geometry into its constituent categories were presented. Then, the relationship between Hyperbolic Geometry and Graph Geometry was established, highlighting the reason why the former fully characterizes the latter. In addition, the PSO method was analyzed together with its variants, which constitute the basis for the HyperMap method, which in turn formed the main pillar of the experiments conducted in this thesis. Finally, an overview of the science of Machine Learning was provided, and more specifically of Neural Networks.

The practical part consists of three sections. First, graph data generated by the Barabasi–Albert method were collected. This was followed by the organization of the Machine Learning experiments, where their framework was defined. In particular, the number of epochs, the learning rate, the architecture, and the optimizer were selected. According to these choices, the main experiments of the thesis were conducted. First, the model's performance on embeddings for graphs of size 100 nodes was examined, and these embeddings were then compared with the corresponding ones produced by the HyperMap method. These experiments were followed by experiments on larger graphs (100, 200, 300, and 400 nodes), which were compared both with the HyperMap method and with the model's performance in the initial experiments. In this way, the existence of a trend in the model's performance with respect to graph size was also investigated.

5.2 Conclusions

In the first experiments, namely those concerning graphs of size 100, the analysis led to the conclusion that the model approximates the HyperMap method sufficiently well. The number of Communities, as well as the relative distances between them, are in agreement with HyperMap when HyperMap identifies clearly distinct Communities. When HyperMap presents a more spread-out distribution, and therefore an inability to form Communities, the model diverges and produces Communities of its own. Finally, regarding the hyperbolic distances, the minimization of which is also the main objec-

tive of HyperMap, the model produced distributions in which more pairs of nodes were characterized by large hyperbolic distances.

In the experiments concerning the embeddings of the larger graphs, a very encouraging observation emerged. More specifically, in the analysis of the hyperbolic-distance distributions, the number of pairs characterized by large hyperbolic distances was smaller and inversely proportional to graph size. In addition, the maximum hyperbolic distance appeared reduced in comparison with the HyperMap method. Finally, the analysis of the angular-coordinate distributions for the larger graphs has the same characteristics as that for the graphs of size 100.

5.3 Future Extensions

After the completion of all the experiments conducted within the framework of this thesis, three main directions were identified that could improve or extend the work that was carried out.

First, the experiments were conducted on graphs of size up to 400 nodes. Increasing the graph size may further clarify the trend in the model's performance in greater depth. The goal is to test graphs of size up to 150000 nodes, for example the structure of the Internet, as is also done in the research works mentioned in the bibliography.

Moreover, it is a fact that real graph data were not used in the experiments of this thesis, but only synthetic ones. It would be interesting to evaluate the performance of this methodology on graphs that do not belong to the Scale-Free category and to investigate the reasons that would affect, or not affect, the performance of the model.

Finally, the architectures that were studied are not characterized by many layers. A process of enriching the models by making them deeper would provide knowledge about the complexity of embedding graphs in Hyperbolic Spaces, since it is known that in more complex problems, increasing model depth also improves performance.

Bibliography

[1] *Medium*. https://ai.plainenglish.io/social-network-analysis-social-circles-of-facebook-611877849d59. Access date: 29-6-2025.

[2] *GitHub*. https://github.com/XinyueTan/Social-Network-Analysis-?tab=readme-ov-file. Access date: 15-5-2025.

[3] Kang Cao και Yan Zhang. *Urban planning in generalized non-Euclidean space*. *Planning Theory*, 12:335–350, 2013.

[4] *WikiBooks*. https://en.m.wikibooks.org/wiki/Geometry/Hyperbolic_and_Elliptic_Geometry/. Access date: 13-5-2025.

[5] *Wikipedia*. https://en.wikipedia.org/wiki/Poincar%C3%A9_disk_model. Access date: 28-6-2025.

[6] *Medium*. https://medium.com/@ngneha090/a-guide-to-supervised-learning-f2ddf1018ee0. Access date: 29-6-2025.

[7] *SuperAnnotate*. https://www.superannotate.com/blog/supervised-learning-and-other-machine-learning-tasks. Access date: 29-6-2025.

[8] *Linkedin*. https://www.linkedin.com/pulse/perceptron-the-basic-building-block-neural-networks-ayush-meharkure-p46if. Access date: 29-6-2025.

[9] *Machine Learning Geek*. https://machinelearninggeek.com/multi-layer-perceptron-neural-network-using-python/. Access date: 14-5-2025.

[10] Thomas Bläsius, Tobias Friedrich και Maximilian Katzmann. *Efficiently Approximating Vertex Cover on Scale-Free Networks with Underlying Hyperbolic Geometry*. *Algorithmica*, 85:1–34, 2023.

[11] Vasileios Karyotis, Eleni Stai και Symeon Papavassiliou. *Evolutionary Dynamics of Complex Communications Networks*. CRC Press, 2013.

[12] Dehua Peng, Zhipeng Gui και Huayi Wu. *Interpreting the Curse of Dimensionality from Distance Concentration and Manifold Effect*, 2025.

[13] Bryan Perozzi, Rami Al-Rfou και Steven Skiena. *DeepWalk: online learning of social representations*. *Proceedings of the 20th ACM SIGKDD international conference on Knowledge discovery and data mining*, KDD '14, σελίδα 701–710. ACM, 2014.

[14] Aditya Grover και Jure Leskovec. *node2vec: Scalable Feature Learning for Networks*, 2016.

[15] Stephen W Hawking και George FR Ellis. *The Large Scale Structure of Space-Time*. Cambridge Monographs on Mathematical Physics. Cambridge University Press, 1973.

[16] . ί . ύ. *μώ ή μί μώ ώ*, 2009.

[17] Dmitri Krioukov, Fragkiskos Papadopoulos, Maksim Kitsak, Amin Vahdat και Marián Boguñá. *Hyperbolic geometry of complex networks*. *Physical Review E*, 82(3), 2010.

[18] Hao Jiang, Lixia Li, Yuanyuan Zeng, Jiajun Fan και Lijuan Shen. *Low-Complexity Hyperbolic Embedding Schemes for Temporal Complex Networks*. *Sensors*, 22(23), 2022.

[19] Gregorio Alanis-Lobato, Pablo Mier και Miguel Andrade. *Efficient embedding of complex networks to hyperbolic space via their Laplacian*. *Scientific Reports*, 6:30108, 2016.

[20] Bianka Kovács και Gergely Palla. *Optimisation of the coalescent hyperbolic embedding of complex networks*. *Scientific Reports*, 11:8350, 2021.

[21] Guillermo García-Pérez, Antoine Allard, M.Ángeles Serrano και Marián Boguñá. *Mercator: uncovering faithful hyperbolic embeddings of complex networks*, 2019.

[22] Fragkiskos Papadopoulos, Constantinos Psomas και Dmitri Krioukov. *Network Mapping by Replaying Hyperbolic Growth*. *IEEE/ACM Transactions on Networking*, 23(1):198–211, 2015.

[23] Martin Keller-Ressel και Stephanie Nargang. *Hydra: A method for strain-minimizing hyperbolic embedding of network- and distance-based data*, 2019.

[24] Martin Keller-Ressel και Stephanie Nargang. *Strain-Minimizing Hyperbolic Network Embeddings with Landmarks*, 2022.

[25] Fan Zhou, Ce Li, Xovee Xu, Leyuan Liu και Goce Trajcevski. *HGENA: A Hyperbolic Graph Embedding Approach for Network Alignment*. σελίδες 1–6, 2021.

[26] Fragkiskos Papadopoulos, Maksim Kitsak, M. Ángeles Serrano, Marián Boguñá και Dmitri Krioukov. *Popularity versus similarity in growing networks*. *Nature*, 489(7417):537–540, 2012.

[27] Miller Mcpherson, Lynn Smith-Lovin και James Cook. *Birds of a Feather: Homophily in Social Networks*. *Annual Review of Sociology*, 27:415–, 2001.

[28] Ozgür Simşek και David Jensen. *Navigating networks by using homophily and degree*. *Proceedings of the National Academy of Sciences of the United States of America*, 105:12758–62, 2008.

[29] S. N. Dorogovtsev. *Lectures on Complex Networks*. *Oxford: Oxford University Press*, 2010.

[30] Pedro Domingos. *A Few Useful Things to Know About Machine Learning*. *Commun. ACM*, 55:78–87, 2012.

[31] Keiron O'Shea και Ryan Nash. *An Introduction to Convolutional Neural Networks*, 2015.

[32] Jie Zhou, Ganqu Cui, Shengding Hu, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li και Maosong Sun. *Graph Neural Networks: A Review of Methods and Applications*, 2021.

[33] William L. Hamilton, Rex Ying και Jure Leskovec. *Inductive Representation Learning on Large Graphs*, 2018.

[34] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò και Yoshua Bengio. *Graph Attention Networks*, 2018.

[35] *NetworkX*. https://networkx.org/. Access date: 29-5-2025.

[36] T.M. Cover και J.A. Thomas. *Elements of Information Theory*. Wiley, 2006.

[37] Martin Arjovsky, Soumith Chintala και Léon Bottou. *Wasserstein GAN*, 2017.

[38] Gabriel Peyré και Marco Cuturi. *Computational Optimal Transport*, 2020.

[39] Muhammad Ali Masood και Rabeeh Ayaz Abbasi. *Using graph embedding and machine learning to identify rebels on twitter*. *Journal of Informetrics*, 15(1):101121, 2021.

[40] Eleni Stai, Vasileios Karyotis και Symeon Papavassiliou. *A hyperbolic space analytics framework for big network data and their applications*. *IEEE Network*, 30(1):11–17, 2016.

[41] Octavian Eugen Ganea, Gary Bécigneul και Thomas Hofmann. *Hyperbolic Neural Networks*, 2018.

[42] Maximilian Nickel και Douwe Kiela. *Poincaré Embeddings for Learning Hierarchical Representations*, 2017.

[43] Nino Arsov και Georgina Mirceva. *Network Embedding: An Overview*, 2019.

[44] Mengjia Xu. *Understanding graph embedding methods and their applications*, 2020.

[45] Vasileios Karyotis, Konstantinos Tsitseklis, Konstantinos Sotiropoulos και Symeon Papavassiliou. *Big Data Clustering via Community Detection and Hyperbolic Network Embedding in IoT Applications*. *Sensors*, 18(4), 2018.

[46] Wei Peng, Tuomas Varanka, Abdelrahman Mostafa, Henglin Shi και Guoying Zhao. *Hyperbolic Deep Neural Networks: A Survey*, 2021.

[47] S. He, S. Xiong, Y. Ou, J. Zhang, J. Wang, Y. Huang και Y. Zhang. *An Overview on the Application of Graph Neural Networks in Wireless Networks*, 2021.

[48] Menglin Yang, Min Zhou, Marcus Kalander, Zengfeng Huang και Irwin King. *Discrete-time Temporal Network Embedding via Implicit Hierarchical Learning in Hyperbolic Space*. *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, KDD '21, σελίδα 1975–1985. ACM, 2021.

[49] Eleni Stai, Konstantinos Sotiropoulos, Vasileios Karyotis και Symeon Papavassiliou. *Hyperbolic Embedding for Efficient Computation of Path Centralities and Adaptive Routing in Large-Scale Complex Commodity Networks*. *IEEE Transactions on Network Science and Engineering*, 4(3):140–153, 2017.

[50] Xiao Wang, Deyu Bo, Chuan Shi, Shaohua Fan, Yanfang Ye και Philip S. Yu. *A Survey on Heterogeneous Graph Embedding: Methods, Techniques, Applications and Sources*, 2020.

[51] Carlo Cannistraci, Gregorio Alanis-Lobato και Timothy Ravasi. *Minimum curvilinearity to enhance topological prediction of protein interactions by network embedding*. *Bioinformatics (Oxford, England)*, 29:i199–i209, 2013.

[52] Alessandro Muscoloni, Josephine Thomas, Sara Ciucci, Ginestra Bianconi και Carlo Cannistraci. *Machine learning meets complex networks via coalescent embedding in the hyperbolic space*. *Nature Communications*, 8, 2017.

[53] Marián Boguñá, Fragkiskos Papadopoulos και Dmitri Krioukov. *Sustaining the Internet with hyperbolic mapping*. *Nature Communications*, 1(1), 2010.

[54] Konstantinos Tsitseklis, Maria Krommyda, Vasileios Karyotis, Verena Kantere και Symeon Papavassiliou. *Scalable Community Detection for Complex Data Graphs via Hyperbolic Network Embedding and Graph Databases*. *IEEE Transactions on Network Science and Engineering*, 8(2):1269–1282, 2021.

[55] Joshua B. Tenenbaum, Vinde Silva και John C. Langford. *A Global Geometric Framework for Nonlinear Dimensionality Reduction*. *Science*, 290(5500):2319–2323, 2000.

[56] Zhilin Yang, William W. Cohen και Ruslan Salakhutdinov. *Revisiting Semi-Supervised Learning with Graph Embeddings*, 2016.

[57] Anthony Baptista, Rubén J. Sánchez-García, Anaïs Baudot και Ginestra Bianconi. *Zoo Guide to Network Embedding*, 2023.

[58] Eleni Stai, Konstantinos Sotiropoulos, Vasileios Karyotis και Symeon Papavassiliou. *Hyperbolic Traffic Load Centrality for large-scale complex communications networks*. *2016 23rd International Conference on Telecommunications (ICT)*, σελίδες 1–5, 2016.

[59] Matteo Bruno, Sandro Ferreira Sousa, Furkan Gursoy, Matteo Serafino, Francesca V. Vianello, Ana Vranić και Marián Boguñá. *Community Detection in the Hyperbolic Space*, 2019.

[60] Robert Jankowskiand Pegah Hozhabrierdi and Marián Boguñá and M. Ángeles Serrano. *Feature-aware ultra-low dimensional reduction of real networks*, 2024.

[61] Marián Boguñá, Ivan Bonamassa, Manlio De Domenico, Shlomo Havlin, Dmitri Krioukov και M. Ángeles Serrano. *Network geometry*. *Nature Reviews Physics*, 3(2):114–135, 2021.

[62] Nikolaos Papadis, Eleni Stai και Vasileios Karyotis. *A path-based recommendations approach for online systems via hyperbolic network embedding*. *2017 IEEE Symposium on Computers and Communications (ISCC)*, σελίδες 973–980, 2017.

[63] Jaspervan der Kolk, M. Ángeles Serrano και Marián Boguñá. *Random graphs and real networks with weak geometric coupling*. *Physical Review Research*, 6(1), 2024.

[64] Muhua Zheng, Guillermo García-Pérez και Marián Boguñáand M. Ángeles Serrano. *Scaling up real networks by geometric branching growth*, 2020.

[65] Marián Boguñá, Dmitri Krioukov, Pedro Almagro και M. Ángeles Serrano. *Small worlds and clustering in spatial networks*. *Physical Review Research*, 2(2), 2020.

[66] Robert Jankowski, Antoine Allard, Marián Boguñá και M.Ángeles Serrano. *The D-Mercator method for the multidimensional hyperbolic embedding of real networks*, 2023.

[67] Vasileios Karyotis, Konstantinos Tsitseklis, Konstantinos Sotiropoulos και Symeon Papavassiliou. *Enhancing Community Detection for Big Sensor Data Clustering via Hyperbolic Network Embedding*. *2018 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*, σελίδες 266–271, 2018.

[68] Fragkiskos Papadopoulos, Dmitri Krioukov, Marián Boguñá και Amin Vahdat. *Greedy Forwarding in Dynamic Scale-Free Networks Embedded in Hyperbolic Metric Spaces*. *2010 Proceedings IEEE INFOCOM*, σελίδες 1–9. IEEE, 2010.

[69] Gregorio Alanis-Lobato, Pablo Mier και Miguel Andrade. *Manifold learning and maximum likelihood estimation for hyperbolic network embedding*. *Applied Network Science*, 1, 2016.